

Klasifikasi URL Berbahaya dengan Neural Network

SKRIPSI

Diajukan Untuk Memenuhi Salah Satu Syarat

Memperoleh Gelar Sarjana Komputer

Program Studi Informatika



Diajukan oleh:

Mathys Jorge Alberino Seilatu

205314714

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS SANATA DHARMA

YOGYAKARTA

2023

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
DAFTAR LAMPIRAN	v
ABSTRAK	vi
BAB I PENDAHULUAN.....	1
1.1. Latar Belakang.....	1
1.2. Rumusan Masalah.....	2
1.3. Batasan Masalah	2
1.4. Tujuan Penelitian	3
1.5. Manfaat Penelitian	3
BAB II TINJAUAN PUSTAKA.....	4
2.1. Landasan Teori	4
2.1.1. URL	4
2.1.2. Convolutional Neural Network	5
2.1.3. Word Embedding	6
2.1.4. Tokenizing	6
2.1.5. Long Short Term Memory (LSTM).....	7
2.1.6. K-Fold Cross Validation	8
2.2. Tinjauan Pustaka.....	9
BAB III METODE PENELITIAN	15
3.1. Deskripsi Data	15
3.2. Kebutuhan Perangkat Hardware dan Software	16
3.3. Perancangan Sistem.....	17
3.4. Skema Pengujian.....	18
3.4.1. Cleaning dan Normalize	18
3.4.2. Tokenize	18
3.4.3. Pengujian Data dalam Model.....	19
3.4.4. Evaluasi	21
3.5. Desain User Interface.....	21
3.6. Jadwal Penelitian	22
BAB IV HASIL PENELITIAN DAN PEMBAHASAN.....	23
4.1 Hasil Skenario Pengujian	23

4.2	Analisa Hasil Pengujian	24
4.2.1	Pengujian Tanpa K-Fold	24
4.2.2	Pengujian Dengan K-Fold	29
BAB V	KESIMPULAN DAN SARAN	33
5.1.	Kesimpulan	33
5.2.	Saran	34
DAFTAR PUSTAKA		35
LAMPIRAN		36

DAFTAR GAMBAR

Gambar 2.1.1.1 Struktur Tautan.....	4
Gambar 2.1.2.1 Use-case tiap Neural Network	5
Gambar 2.1.2.2 CNN 1D	5
Gambar 2.1.2.3 CNN 1D 2	6
Gambar 2.1.3.1 Ilustrasi Word Embedding.....	6
Gambar 2.1.4.1 Tokenizing.....	7
Gambar 2.1.5.1 Arsitektur LSTM	7
Gambar 2.1.6.1 Diagram Perancangan Sistem	17
Gambar 3.4.2.1 sentence to token	19
Gambar 3.4.2.2 token to siquence.....	19
Gambar 3.4.3.1 Gambaran Kasar model.....	20
Gambar 3.4.4.1 Rancangan Desain GUI.....	21
Gambar 4.2.1.1 Grafik Akurasi Arsitektur 3LSTM 1CNN	24
Gambar 4.2.1.2 Grafik Akurasi Arsitektur 2LSTM 2CNN.....	25
Gambar 4.2.1.3 Grafik Akurasi Arsitektur 1LSTM 3CNN	25
Gambar 4.2.1.4 Grafik Akurasi Arsitektur 4CNN.....	26
Gambar 4.2.1.5 Grafik Akurasi Arsitektur 4LSTM	26
Gambar 4.2.1.6 Grafik Akurasi Arsitektur 2LSTM 1CNN.....	27
Gambar 4.2.1.7 Grafik Rata Rata Akurasi Learning Rate.....	27
Gambar 4.2.1.8 Grafik Heatmap Korelasi Tanpa K-Fold	28
Gambar 4.2.2.1 Akurasi Data Baru dengan K-Fold.....	29
Gambar 4.2.2.2 Akurasi Data Latih dengan K-Fold	29
Gambar 4.2.2.3 Pengujian Pertama Crash	29
Gambar 4.2.2.4 Pengujian dengan Melakukan Fold Satu Persatu	30
Gambar 4.2.2.5 Grafik Heatmap Korelasi K-Fold.....	30
Gambar 4.2.2.6 Grafik Persebaran Puncak Akurasi Tertinggi	31

DAFTAR TABEL

Tabel 2.2.1 Tabel tinjauan pustaka.....	9
Tabel 3.1.1 Contoh Data Mentah	15
Tabel 3.4.1 Data Setelah Normalisasi dan Cleaning	18
Tabel 3.4.2 Uji Coba	20
Tabel 4.1.1 Hasil Uji dengan K-Fold	23
Tabel 4.1.2 Hasil Uji tanpa K-Fold	23

DAFTAR LAMPIRAN

Lampiran 1 Tahapan Pengambilan Data	36
Lampiran 3 Tahapan Balancing	38
Lampiran 4 Tahapan Label Encoding	39
Lampiran 5 Tahapan Tokenisasi.....	39
Lampiran 6 Tahapan Cleaning	43
Lampiran 7 K-Fold Cross Validation	44
Lampiran 8 Tahapan Optimalisasi.....	45
Lampiran 9 Tabel Percobaan dengan penerapan optimalisasi	46
Lampiran 10 Tabel Percobaan tanpa penerapan optimalisasi	46
Lampiran 11 Proses dalam Layer Embedding	47
Lampiran 12 Hasil Matriks Embedding layer dengan output 2	47
Lampiran 13 Hasil Matriks Embedding layer dengan output 4	48
Lampiran 14 Proses dalam layer CNN	49
Lampiran 15 Hasil CNN dengan filter 4.....	51
Lampiran 16 Hasil CNN dengan filter 16.....	51
Lampiran 17 Proses dalam Layer LSTM.....	52
Lampiran 18 Seluruh Data Pengujian tanpa K-Fold.....	54
Lampiran 19 Seluruh Data Pengujian Menggunakan K-Fold.....	56
Lampiran 20 Arsitektur Model Terbaik Pengujian K-Fold.....	58
Lampiran 21 Grafik akurasi dan loss tiap iterasi 3-fold.....	59
Lampiran 22 Grafik akurasi dan loss tiap iterasi 5-fold.....	60
Lampiran 23 Grafik akurasi dan loss tiap iterasi 7-fold.....	61
Lampiran 24 Grafik akurasi dan loss tiap iterasi 9-fold.....	62
Lampiran 25 Error yang didapat tiap run.....	63

ABSTRAK

Tautan keamanan web semakin bertambah setiap harinya, dan deteksi serta analisis halaman web yang berbahaya semakin penting dan sulit dilakukan, *Phishing* adalah salah satu bentuk kejahatan dengan cara mendapatkan informasi sensitif dengan meniru pengirim yang terpercaya di dalam saluran komunikasi, jika dalam dunia internet berarti meniru *domain name* dari sebuah *website* sehingga pengguna internet tertipu dan akhirnya memberikan data pribadi mereka.

Dikarenakan perkembangan teknologi dan membludaknya *URL* (tautan) di internet maka kita tidak dapat melakukan pengecekan satu persatu terhadap tautan tautan yang ada, maka dibangunlah sistem untuk membantu dalam hal tersebut.

Prediksi *URL* berbahaya dibangun menggunakan *Deep Learning* dengan metode campuran *Long Short term Memory* (LSTM) dan *Convolutional Neural Network* (CNN). Pelatihan dan pengujian model tersebut dilakukan pada platform *Kaggle Notebook*. Setelah selesai, model akan dievaluasi akurasi dan f1 skornya.

Kata kunci : tautan berbahaya, *neural network*, *phishing*, *fraud detection*, *kaggle notebook*.

BAB I

PENDAHULUAN

1.1. Latar Belakang

Dengan perkembangan teknologi internet yang terus berlangsung, tautan keamanan *web* semakin bertambah setiap harinya, dan deteksi serta analisis halaman *web* yang berbahaya semakin penting dan sulit dilakukan [1]. *Phishing* adalah salah satu bentuk kejahatan dengan cara mendapatkan informasi sensitif dengan meniru pengirim yang terpercaya di dalam saluran komunikasi. *Phishing* sering digunakan untuk mendapatkan informasi atau uang dengan cara yang tidak benar. Biasanya, pesan yang dikirimkan mengandung *software* atau tautan berbahaya. Lebih mudah untuk membuat laman palsu daripada membobol keamanan sistem. Selain itu, *phishing* dapat diluncurkan dengan biaya yang relatif murah dari mana saja di dunia karena internet yang terbuka dan anonim. Laman *phishing* berisi logo tipe dan teks yang ditujukan untuk meniru laman yang asli [2].

Dikarenakan maraknya hal tersebut menjadikan jenis *fraud* ini mulai berkembang dan diterapkan pada halaman *website* dan ketika diakses melalui URL, halaman dengan *phishing* tersebut bisa menyebabkan masalah bagi pengguna, mulai dari menginfeksi komputer pribadi dengan *malware*, mengunduh perangkat lunak mata-mata, mencuri kredensial, hingga pencurian uang dari kartu bank [3]. Salah satu cara paling baik untuk mengategorikan URL yang berbahaya atau tidak adalah dengan *Machine Learning*.

Jaringan Syaraf Tiruan (Neural Network) sering disebut juga *Deep Learning* dan semakin populer dalam penelitian saat ini khususnya dalam klasifikasi teks. *Neural Network* telah menunjukkan kinerja yang sangat baik dalam klasifikasi teks [4]. Bukti yang kuat dapat ditemukan melalui beberapa penelitian yang menunjukkan bahwa penggunaan *Neural Network* dapat menghasilkan akurasi yang sangat tinggi, bahkan melebihi 95% [5]. Sehingga *Neural Network* dapat digunakan untuk mendeteksi URL berbahaya dengan cukup akurat.

1.2. Rumusan Masalah

1. Dapatkah *Neural Network* digunakan untuk mengklasifikasikan url dengan baik?
2. *Variable* apa yang mempengaruhi akurasi dari model yang telah dibuat?

1.3. Batasan Masalah

1. Metode yang digunakan adalah *Deep Learning Neural Network* dengan komposisi layer *word embedding*, *convolutional*, *max pooling*, *LSTM*(*Long Short-Term Memory*), kemudian dilakukan modifikasi *hyper-parameter* dan komposisi *layer* untuk mencari model dengan akurasi yang tinggi.
2. Data yang digunakan adalah data dari *website* pencatat *link* berbahaya atau *website* penyedia data dan kemudian dikombinasikan dengan *website* yang masuk dalam *white list* atau *website* yang sering kita gunakan seperti *google.com*, *youtube.com*, dan sebagainya.

1.4. Tujuan Penelitian

1. Untuk mengetahui komposisi *layer Neural Network* apa yang dapat digunakan untuk mengklasifikasikan *URL* berbahaya dengan baik.
2. Untuk mengetahui *hyper-parameter* yang paling optimal untuk klasifikasi *URL*.

1.5. Manfaat Penelitian

1. Mengurangi ancaman kejahatan *cyber* bagi pengguna awam.
2. Peningkatan keamanan informasi *online*.

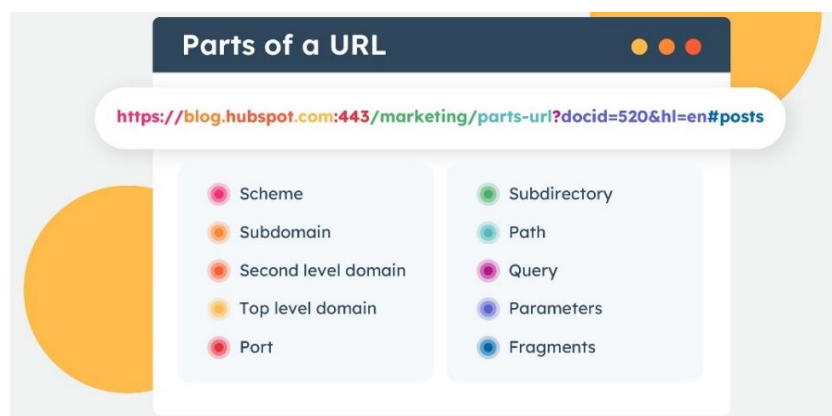
BAB II

TINJAUAN PUSTAKA

2.1. Landasan Teori

2.1.1. URL

URL (*Uniform Resource Locator*) atau tautan atau *link* adalah koneksi antara *resource* satu ke lainnya. Awal mula dari tautan adalah World Wide Web (WWW) yang berfungsi untuk melihat multimedia-based dokumen yang menampilkan dokumen dengan teks, gambar, animasi atau video. World Wide Web Consortium (W3C) adalah organisasi yang menstandarisasi perihal HyperText Markup Language (XHTML), Cascading Style Sheets (CSS), HyperText Markup Language (HTML) dan Extensible Markup Language (XML). Untuk dapat melihat dokumen yang dipublikan dalam internet adalah dengan cara mengakses alamat URL-nya (URL address), struktur dasar dari URL terdiri dari protokol, *subdomain*, nama *domain*, dan ekstensinya[6].



Gambar 2.1.1.1 Struktur Tautan (*https://blog.hubspot.com/marketing/parts-url*)

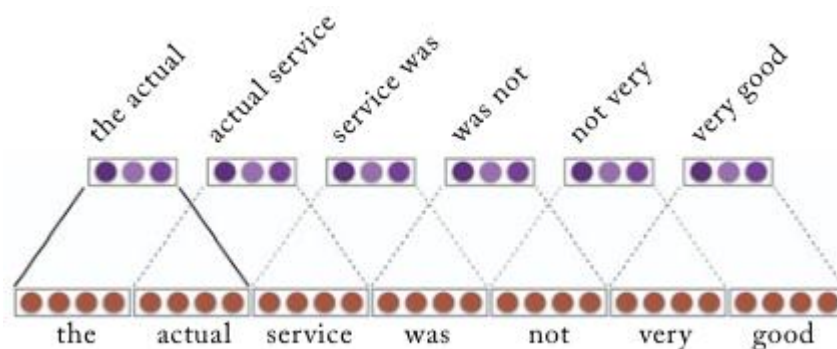
2.1.2. Convolutional Neural Network

CNN adalah tipe lain dari *Deep Neural Network*, jika menggunakan CNN biasanya disertai juga penggunaan *pooling layer*, dimana *pooling layer* tersebut berfungsi untuk membagi region dari *convolutional* menjadi *subregion* agar komputasi menjadi lebih ringan. CNN sangat sukses dalam kasus klasifikasi gambar [7] tetapi bisa juga digunakan dalam natural *language processing* [8].

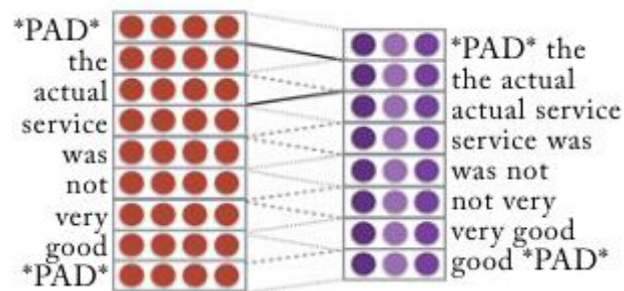
	Recurrent Network	Recursive Neural Tensor Network	Deep Belief Network	Convolution Network	MLP
Text Processing	✓	✓		✓	
Image Recognition			✓	✓	
Object Recognition		✓		✓	
Speech Recognition	✓				
Time Series Analysis	✓				
Classification			✓	✓	✓

Gambar 2.1.2.1 Use-case tiap Neural Network (T. Oliver, 2020)

Dalam klasifikasi teks, input akan diambil dari *embedding layer* kemudian di proses oleh *convolution layer* dengan perlakuan mirip seperti pemrosesan citra, yaitu dengan melakukan *filtering* pada tiap *window* terhadap *convolutional filter* (*kernel*) [9].



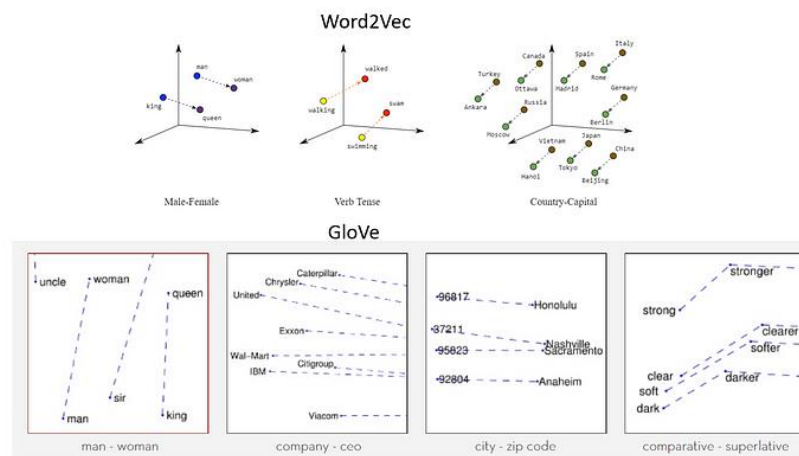
Gambar 2.1.2.2 CNN 1D (Graeme Hirst, dkk, 2017)



Gambar 2.1.2.3 CNN 1D 2 (Graeme Hirst, dkk, 2017)

2.1.3. Word Embedding

Word Embedding adalah Layer wajib untuk teks *processing*, karena pada layer ini tiap kalimat akan dikonversi menjadi vector dengan bobot yang dapat diproses oleh *computer* saat *training* [7].



Gambar 2.1.3.1 Ilustrasi Word Embedding (<https://towardsdatascience.com/word-embeddings-for-nlp-5b72991e01d4>)

2.1.4. Tokenizing

Tokenization adalah memecah kalimat menjadi kata kemudian melabeli kalimat tersebut dengan nomor agar dapat dikomputasikan [10]. Kemudian setelah dipecah pecah, nomor dari kata tersebut disusun ulang sesuai dengan susunan

kalimat awalnya kedalam list (*teks to squence*), setelah selesai kalimat kalimat dalam bentuk list tersebut di samakan dimensinya dan dimasukan kedalam list yang sama (*padding*).

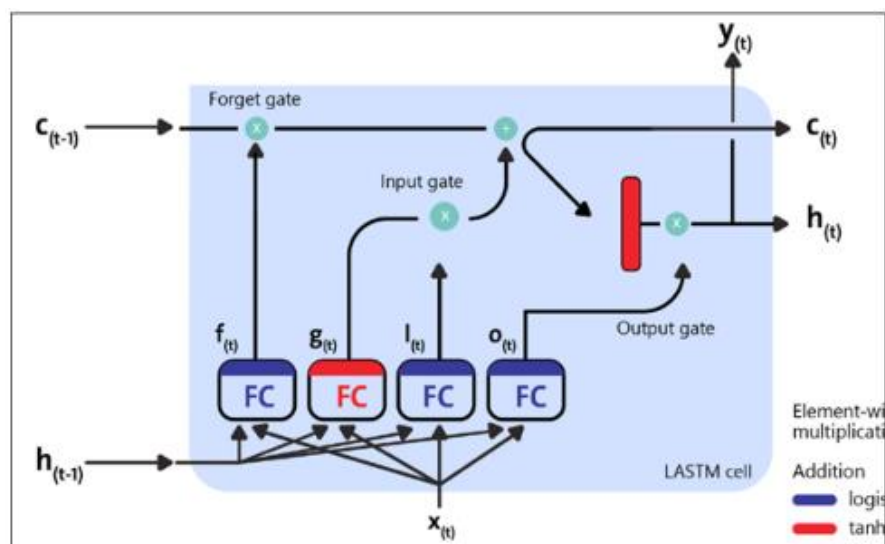
Output:

	I	enjoying	feel	for	freedom	happy	live	love	our	the	we
doc1	1	0	0	0	1	2	0	1	1	1	2
doc2	0	1	1	1	0	1	1	0	0	0	0

Gambar 2.1.4.1 Tokenizing (Alexandra George, 2018)

2.1.5. Long Short Term Memory (LSTM)

LSTM adalah salah satu tipe dari RNN yang ideal untuk digunakan dalam prediksi dan kalsifikasi untuk temporal *sequences*. Pada tiap blok LSTM berisikan tiga tipe *gate*: *input gate*, *output gate*, dan *forget gate* yang mengimplementasikan fungsi tulis, baca, dan reset dalam *cell memory*. *Gate* tesebut bertipe analog dan biasanya digunakan *activation layer sigmoid* [7].



Gambar 2.1.5.1 Arsitektur LSTM (G. Aurélien, 2019)

2.1.6. K-Fold Cross Validation

K-Fold Cross Validation adalah salah satu metode evaluasi kinerja model dengan cara membagi dataset menjadi k subset dengan ukuran yang sama, model dilatih pada k-1 subset dan diuji pada subset ke-k dan diulang sebanyak k, sehingga setiap subset sekali menjadi set uji. Performa K-Fold Cross Validation diukur dengan menghitung rata-rata dari hasil pengujian [11]. Rumus yang digunakan adalah:

$$Akurasi = \frac{Jumlah\ prediksi\ benar}{Total\ prediksi} \quad (2.1.6.1)$$

$$Presisi = \frac{TP}{TP + FP} \quad (2.1.6.2)$$

TP = banyak dari benar diprediksi benar

FP = banyak dari benar diprediksi salah.

$$Recall = \frac{TP}{TP + FN} \quad (2.1.6.3)$$

FN = banyak dari salah diprediksi benar.

Presisi dan *Recall* sering digabung menjadi satu metrik bernama *F1 Score* yang memberikan simplifikasi dengan *harmonic mean* dari *Presisi* dan *Recall* yang berarti *F1 Score* akan tinggi jika nilai dari *Presisi* dan *Recall* juga tinggi [12].

$$F1 = \frac{2 * TP * Recall}{TP + Recall} \quad (2.1.6.4)$$

FP = banyak dari salah diprediksi salah.

2.2. Tinjauan Pustaka

Tabel 2.2.1 Tabel tinjauan pustaka

Peneliti, Judul, Penerbit dan Tahun Terbit	Tujuan penelitian	Data dan Hasil
Judul: URLDeep: Continuous Prediction of Malicious URL with Dynamic Deep Learning in Social Networks Peneliti: Putra Wanda and Huang Jin Jie Penerbit: International Journal of Network Security Tahun Terbit: 2019	Penelitian ini ditujukan untuk menguji teknik baru Neural Network Dynamic CNN yaitu URLDeep untuk membedakan link berbahaya dan tidak, kemudian mendapat titingkat akurasi yang tinggi dalam mendeteksi URL berbahaya secara kontinu dan dinamis di lingkungan jaringan sosial.	Populasi: URL dari VirusTotal and PhishTank Sample: 16 juta terdiri dari 14,050,275 (Benign) dan 1.049,725 (Malicious) Pengukuran: akurasi (accuracy), presisi (precision), area under the curve (AUC), dan waktu komputasi. Alat Analisis: Teknik deep learning berbasis LSTM (Long Short-Term Memory) dan transfer learning untuk membangun model prediksi URL berbahaya secara kontinu dan dinamis. Transfer learning merupakan teknik yang digunakan untuk memanfaatkan model yang sudah dilatih sebelumnya pada masalah serupa untuk mempercepat proses pembelajaran pada masalah yang baru. Hasil: Hasil penelitiannya adalah bahwa pengembangan metode deteksi URL berbahaya yang menggunakan teknik deep learning berbasis LSTM dan transfer learning dengan fitur-fitur dinamis seperti yang disebutkan tersebut mampu mencapai tingkat akurasi yang tinggi dalam mendeteksi URL berbahaya secara kontinu dan dinamis di lingkungan jaringan sosial. Selain itu, metode ini juga mampu mengatasi permasalahan klasifikasi URL yang bersifat dinamis dan terus berubah seiring waktu. Walaupun demikian, tidak disebutkan berapa akurasi yang didapat

Judul: Malicious URL Detection Based on Improved Multilayer Recurrent Convolutional Neural Network Model Peneliti: Zuguo Chen, Yanglong Liu, Chaoyang Chen, Ming Lu and Xuzhuo Zhang Penerbit: Security and Communication Networks Tahun Terbit: 2021	Penelitian ini ditujukan untuk memberikan gambaran mengenai potensi dan tantangan penggunaan machine learning dalam keamanan siber, privasi, dan keselamatan publik pada aplikasi yang sedang berkembang, serta untuk mengidentifikasi solusi dan strategi untuk mengatasi tantangan tersebut.	Populasi: URL Sample: 200,000 URL, dibagi menjadi 100,000 normal URL dilabeli "good" dan 100,000 malicious URL dilabeli "bad" training test perbandingan adalah 9:1 Pengukuran: akurasi (accuracy), presisi (precision), recall, area under the curve (AUC), dan waktu komputasi. Alat Analisis: CNN menggunakan YOLO algorithm kemudian dikembangkan menjadi CSPDarknet neural network model, kemudian menggunakan bidirectional LSTM recurrent neural network algorithm untuk ekstraksi fitur Hasil: Hasil penelitiannya adalah bahwa menggunakan metode yang disebutkan menghasilkan efektivitas dan kecepatan deteksi yang lebih baik dan mendapatkan high accuracy, high recall rate, and high accuracy dibanding dengan Text-RCNN, BRNN, dan model lainnya. namun demikian masih diperlukan pengembangan untuk menghindari missing information dari URL teks vectorization. akurasi dengan YOLO CSPDarknet menghasilkan 94% dengan nilai loss value 0.19. kemudian dengan traditional bidirectional recurrent neural network menunjukan indikasi over fitting, dengan RCNN akurasi adalah 92% dengan loss value 0.22, dengan tradisional RNN menghasilkan 90% dengan minimum loss value 0.32, dengan Neural Network models based on fully connected layers menghasilkan 86% dengan loss value 0.33.
--	---	---

Judul: A Malicious URL Detection Model Based on Convolutional Neural Network Peneliti: Zhiqiang Wang, Xiaorui Ren, Shuhao Li, Bingyan Wang, Jianyi Zhang, dan Tao Yang Penerbit: Security and Communication Networks Tahun Terbit: 2021	Tujuan penelitian adalah untuk mendesain sebuah model deteksi URL jahat berbasis Convolutional Neural Network yang efektif dan sulit dihindari oleh penyerang. Model ini menggunakan metode embedding kata berbasis embedding karakter untuk mempelajari representasi vektor URL dan menggabungkan layer konvolusi dan pooling untuk mengekstraksi fitur URL dengan lebih efisien. Melalui serangkaian eksperimen, penelitian menunjukkan bahwa model ini mampu mendeteksi URL jahat dengan akurasi yang tinggi (98%)	Populasi: URL Sample: Tidak disebutkan Pengukuran: akurasi (accuracy), presisi (precision), recall, F1-score, area under the curve (AUC), dan waktu komputasi. Alat Analisis: Dynamic convolutional neural network (DCNN). memodifikasi pooling layer default menjadi k-max-pooling layer. embedding method Hasil: Penelitian ini bertujuan untuk merancang model deteksi URL berbahaya berbasis deep learning dengan menggunakan metode word embedding berdasarkan karakter embedding. Hasil eksperimen menunjukkan model deteksi yang dirancang efektif dalam mendeteksi URL berbahaya. Namun, dengan berkembangnya internet dan semakin beragamnya jenis URL berbahaya, model ini harus diperbarui secara berkala agar tetap efektif dalam berbagai skenario aplikasi yang kompleks.
---	--	---

Judul: Bidirectional LSTM Malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network Peneliti: Huan-huan Wang & Long Yu & Sheng-wei Tian & Yong-fang Peng & Xin-jun Pei Penerbit: Appl Intell Tahun Terbit: 2019	Tujuan penelitian adalah untuk mengusulkan algoritma bidirectional LSTM (CBIR) yang berbasis pada convolutional neural network (CNN) dan independent recurrent neural network (RNN) untuk meningkatkan akurasi deteksi halaman web jahat. Algoritma ini menggunakan fitur "texture fingerprint" dan fitur vektor kata URL yang dihasilkan oleh alat word2vec, dan meningkatkan akurasi deteksi halaman web jahat dengan menggunakan algoritma CBIR yang diusulkan.	Populasi: URL dari PhishTank dan known URL Sample: 23,652 terdiri dari 10,000 (legitimate) dan 13,652 dari PhishTank (Malicious) Pengukuran: akurasi (accuracy) dan waktu komputasi. Alat Analisis: convolutional neural network (CNN) dan bidirectional LSTM algorithm (CBIR), digunakan pula teknik word embedding dengan memanfaatkan karakter embedding dan word2vec. Hasil: Penelitian ini mendapatkan tingkat akurasi paling besar adalah 97.82%, tetapi setelah menggunakan URL word vector feature akurasi naik menjadi 98.45%. penggunaan algoritma CBIR yang menggabungkan fitur-fitur seperti texture fingerprint, URL word vector, dan static vocabulary dapat meningkatkan akurasi dalam mendeteksi halaman web yang jahat. Hal ini dapat diobservasi melalui hasil eksperimen yang menunjukkan bahwa metode yang diusulkan dalam penelitian ini memiliki hasil yang lebih baik dibandingkan dengan metode-metode lain yang telah diteliti sebelumnya.
---	---	--

Judul: Accurate and Fast URL Phishing Detector: A Convolutional Neural Network Approach Peneliti: Wei Wei, Qiao Ke, Jakub Nowak, Marcin Korytkowski, Rafał Scherer, Marcin Woźniak Penerbit: Computer Networks Tahun Terbit: 2020	Penelitian ini bertujuan untuk mengembangkan metode pendeteksian situs phishing dengan menggunakan jaringan saraf konvolusi yang dapat mengidentifikasi URL yang berbahaya dengan hampir 100% akurasi hanya dengan menganalisis teks URL. Dalam eksperimen, metode CNN yang diusulkan dibandingkan dengan pendekatan lain, terutama dengan jaringan LSTM yang diusulkan dalam penelitian sebelumnya.	Populasi: URL dari PhishTank dan crawling legitimate URL Sample: 21,208 terdiri dari 10,604 (Benign) dan 10,604 (Malicious) Pengukuran: akurasi (accuracy) Alat Analisis: YOLO algorithm, CNN with Embedding Layer, Teknik deep learning berbasis LSTM (Long Short-Term Memory). Hasil: Kesimpulan dari penelitian ini adalah bahwa metode yang diusulkan dapat mengidentifikasi situs phishing dengan akurasi hampir 100% dan mampu mendeteksi serangan zero-day. Selain itu, metode ini lebih cepat dibandingkan dengan pendekatan lain yang menganalisis statistik lalu lintas atau konten web. Metode ini juga dapat digunakan pada perangkat mobile tanpa signifikan mempengaruhi kinerjanya. Jaringan saraf konvolusi yang diusulkan juga lebih mudah dilatih dan lebih cepat konvergensinya dibandingkan dengan jaringan LSTM yang dibandingkan dalam penelitian sebelumnya.
---	---	---

Judul: Detecting Malicious URLs Based on Machine Learning Algorithms and Word Embeddings Peneliti: Andrei Cris,an, Gabriel Florea, Lorand Halasz, Camelia Lemnaru, Ciprian Oprisa Penerbit: IEEE Tahun Terbit: 2020	Mengembangkan model klasifikasi yang handal untuk mendeteksi URL berbahaya dengan memanfaatkan fitur-fitur yang sesuai, baik itu yang sudah ada maupun yang dibangun khusus untuk domain tertentu.	Populasi: URL dari PhishTank dan crawling legitimate URL Sample: approximately 500,000 URL terdiri dari 10:1 untuk clean dan URL berbahaya, kemudian ada lagi 8 millions URL dengan perbandingan 1:1 untuk clean dan URL berbahaya Pengukuran: Precision, accuracy, dan recall rate. Alat Analisis: Kombinasi antara word embeddings dan fitur-fitur domain yang telah dirancang khusus, Teknik oversampling sintetis dan cost-sensitive learning digunakan untuk mengatasi ketidakseimbangan kelas. Teknik-teknik klasifikasi yang berbeda juga dieksplorasi dalam penelitian ini (Cost-Sensitive NN, MLP, Extra Trees).
---	---	---

BAB III

METODE PENELITIAN

3.1. Deskripsi Data

Data yang saya gunakan untuk klasifikasi URL berbahaya adalah data teks yang berasal dari *website* PhishTank (untuk *alternative* saya akan menggunakan data dari Kaggle jika website PhishTank sudah tidak menyediakan data lagi) yang kemudian di-*scraping* untuk URL berbahaya. Kemudian untuk URL yang tidak berbahaya diambil dari whitelist milik hackertarget.com yang diambil dari statistik milik amazon alexa yang berisikan URL umum seperti google.com, youtube.com.

Tabel 3.1.1 Contoh Data Mentah (phishtank.com)

ID	Phish URL	Submitted	Valid?	Online?
8155540	https://kanagawa-u-ac-jp.com/ added on May 25th 2023 7:41 AM	by kubotaa	Unknown	ONLINE
8155539	https://mastersso-kanagawa-u-acweb-jp.com/ added on May 25th 2023 7:40 AM	by kubotaa	Unknown	ONLINE
8155538	https://pepesgifts.vip/ added on May 25th 2023 7:39 AM	by Felix0101	Unknown	ONLINE
8155537	https://paiements-antai-gouv.com/app/pages/index.php... added on May 25th 2023 7:37 AM	by Nameshield	VALID PHISH	ONLINE
8155536	https://paiements-antai-gouv.com/ added on May 25th 2023 7:37 AM	by Nameshield	VALID PHISH	ONLINE
8155535	https://luanajunaiklam.onyx-sites.io/Direct/ujalik/Manage.php... added on May 25th 2023 7:35 AM	by D3Lab	VALID PHISH	ONLINE

Data dari pishtank sudah dila beli dan di uji oleh pengelola *website*-nya dan pelapor *link* untuk *pishing* atau tidaknya, total data yang sudah didapatkan adalah 180.000 data terdiri dari 50% URL berbahaya dan 50% tidak berbahaya

3.2. Kebutuhan Perangkat Hardware dan Software

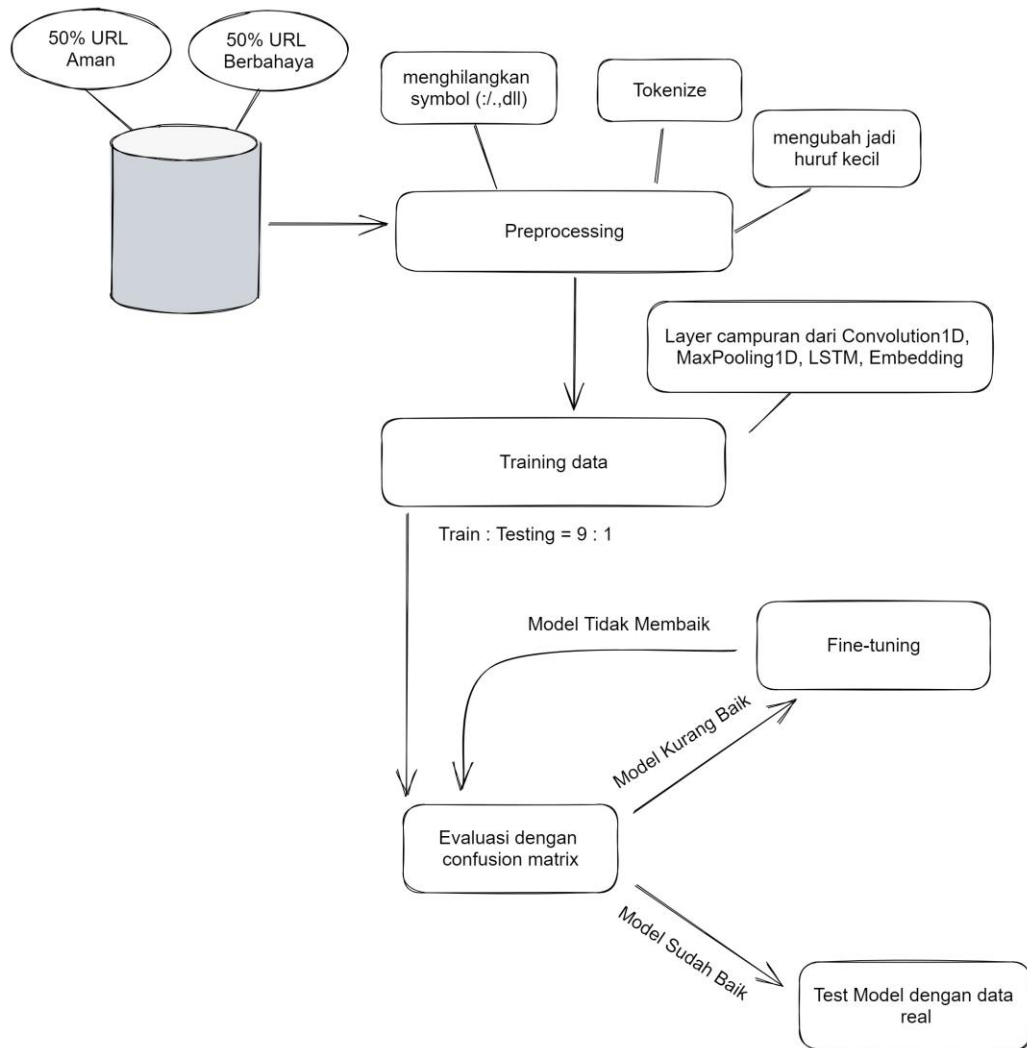
a. Spesifikasi Hardware

- i. Intel(R) Core i5-6330U @ 2.40GHz (4 CPUs) vPro
- ii. RAM 8 GB DDR4 Singel-Channel 2133 Mhz
- iii. Intel(R) HD Graphics 520
- iv. SSD

b. Spesifikasi Software

- i. Sistem Operasi Windows 10 Pro 64-bit
- ii. Browser
- iii. Kaggle Notebook (Jupyter)
- iv. Python 3.*
- v. Library Python;
 - TensorFlow dan Keras
 - Pandas
 - Sklearn
 - Numpy
 - Matplotlib

3.3. Perancangan Sistem



Gambar 2.1.6.1 Diagram Perancangan Sistem

Dari gambar diatas diketahui bahwa data URL *phising* diambil sebanyak 90 ribu dari masing masing kelas (50:50), kemudian data masuk dalam tahap *preprocessing* dengan diberlakukan *cleaning symbol*, mengubah huruf jadi huruf kecil dan terakhir *tokenizing*, setelah itu data masuk kedalam tahap *training* dengan cara pertama *split* data ke *train* dan *test*, kemudian masuk ke model *deep learning* yang terdiri dari *layer Embedding word*, LSTM dan CNN.

3.4. Skema Pengujian

3.4.1. Cleaning dan Normalize

Data yang didapat di *cleaning* dengan menghilangkan *symbol symbol* dan diberlakukan *preprocessing*. Contoh struktur URL:

http://subdomain.domain-name.domain-extension/path-to-resource?parameters

fitur yang digunakan:

- google => domain name
- com => top-lvl domain (domain-extension)
- ../absz/... => path-to-resource

Sehingga menghasilkan:

Tabel 3.4.1 Data Setelah Normalisasi dan Cleaning

URL	Label
kanagawa u ac jp com	No
mastersso kanagawa u acweb jp com	No
pepesgifts vip	No
paiements antai gouv com app pages index php	Phishing
paiements antai gouv com	Phishing
luanajunaiklam onyx sites io direct ujalik Manage php	Phishing

3.4.2. Tokenize

Melakukan tahapan *tokenizing* pada data set yang sudah bersih dan dinormalisasi Menggunakan *library python*. Menambahkan OOV (*Out Of Vocabulary*) sebagai *index 1*.

<OOV>:1	Kanagawa:2	u:3	ac:4	jp:5	com:6	acweb:7	mastersso:8	vip:9
---------	------------	-----	------	------	-------	---------	-------------	-------

Gambar 3.4.2.1 sentence to token

Setelah dilakukan *tokenizing*, mengubah *token* tidak bermakna ke *sequence*, menghasilkan:

kanagawa u ac jp com	mastersso kanagawa u acweb jp com	pepesgifts vip	...
[2, 3, 4, 5, 6]	[8, 1, 3, 7, 5, 6]	[1, 9]	...

Gambar 3.4.2.2 token to squence

Setelah selesai, masuk ke tahap selanjutnya yaitu melakukan *padding sequence* untuk mengubah *array* dengan beda ukuran menjadi satu ukuran yang sama. Menghasilkan:

[[0,2,3,4,5,6],

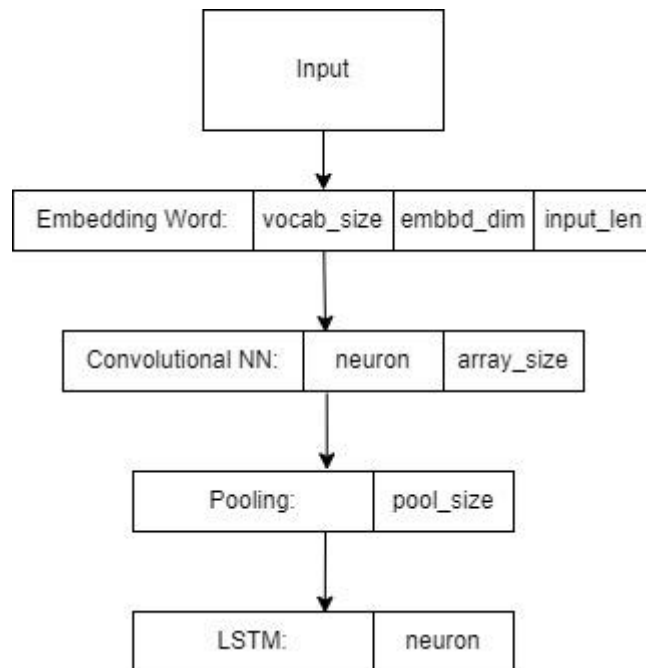
[8,1,3,7,5,6],

[0,0,0,0,1,9]]

Barulah data dapat dimasukan kedalam model.

3.4.3. Pengujian Data dalam Model

Setelah dilakukan *tokenizing*, memasukan data ke model denga *layer* wajib *Embedding Word* dan sisanya dikombinasikan dari CNN dan LSTM.



Gambar 3.4.3.1 Gambaran Kasar model

Modifikasi yang akan dilakukan untuk menemukan model terbaik adalah dengan mengurangi atau menambah jumlah *neuron*, *embedding dimension*, kemudian menambah atau mengurangi jumlah *layer*, dan *learning rate*. Untuk *epoch* akan diberlakukan teknik *early stopping* dimana akan menghentikan *training* jika sudah tidak ada perkembangan akurasi atau *loss* pada beberapa epoch kedepan. Untuk neuron pada tiap layer dipergunakan *randomize* diantara [32, 64, 128, 256, 512] dan untuk learning rate dipergunakan *randomize* diantara [1e-5, 1e-4, 1e-3], kemudian diambil kombinasi yang menghasilkan *validation* akurasi tertinggi.

Tabel 3.4.2 Uji Coba

Layer	Learning rate	Neuron
LSTM, CNN	[1e-5, 1e-4, 1e-3]	[32, 64, 128, 256, 512]
LSTM	[1e-5, 1e-4, 1e-3]	[32, 64, 128, 256, 512]
CNN, Pooling	[1e-5, 1e-4, 1e-3]	[32, 64, 128, 256, 512]

LSTM, CNN, Pooling	[1e-5, 1e-4, 1e-3]	[32, 64, 128, 256, 512]
-----------------------	--------------------	-------------------------

3.4.4. Evaluasi

Pada tahap evaluasi ini akan digunakan *confusion matrix* yang nantinya dijadikan acuan pengambilan keputusan untuk melakukan *re-tuning hyper parameters* dari model atau tidak. Akurasi akan dihitung berdasarkan nilai benar dalam prediksi dibagi nilai sebenarnya dan *f1 score* akan dihitung menggunakan rumus (2.1.6.3)

3.5. Desain User Interface

Predicted as Not a Dangerous URL

Confidence score: 89.23%

Gambar 3.4.4.1 Rancangan Desain GUI

Rancangan desain GUI yang digunakan kurang lebih seperti pada **Gambar**

3.5.1. Untuk cara penggunaanya adalah:

1. *Paste tautan ke form teks*
2. *Kemudian klik tombol submit*
3. *Tunggu sampai confidence score dan prediksi keluar*

3.6. Jadwal Penelitian

No.	Bulan	Kegiatan
1	Juni	Collecting dan Processing Data
2	Juli	Model Implementation
3	Agustus	Model Evaluation
4	September	Pengerjaan Laporan Tugas Akhir
5	Oktober	Pengerjaan Laporan Tugas Akhir
6	November	Pengerjaan Laporan Tugas Akhir

BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

4.1 Hasil Skenario Pengujian

Tabel 4.1.1 Hasil Uji dengan K-Fold

Layer	Learning rate	Neuron	Akurasi
CNN(2), LSTM(2)	0.001	128, 64 & 128, 128	0.82880
CNN(0), LSTM(4)	0.0001	128, 256, 256, 256	0.88647
	0.001	256, 256, 128, 128	0.91724
		128, 256, 128, 128	0.91800
		128, 128, 128, 128	0.91928
		128, 128, 128, 128	0.92388
		128, 256, 128, 256	0.92393
		256, 128, 256, 128	0.92718
		128, 128, 256, 256	0.92827
		128, 128, 128, 128	0.92990
CNN(1), LSTM(3)	0.0001	128 & 128, 256, 128	0.89054
		128 & 128, 128, 128	0.89115
		256 & 128, 256, 128	0.89967
	0.001	128 & 128, 256, 256	0.90714
		128 & 128, 128, 128	0.91115
		128 & 128, 256, 256	0.91500
		128 & 128, 64, 64	0.91501
		256 & 128, 256, 256	0.91700
		128 & 128, 128, 128	0.91760
		128 & 256, 128, 128	0.91948
		256 & 128, 256, 128	0.92108
		128 & 128, 128, 128	0.92302

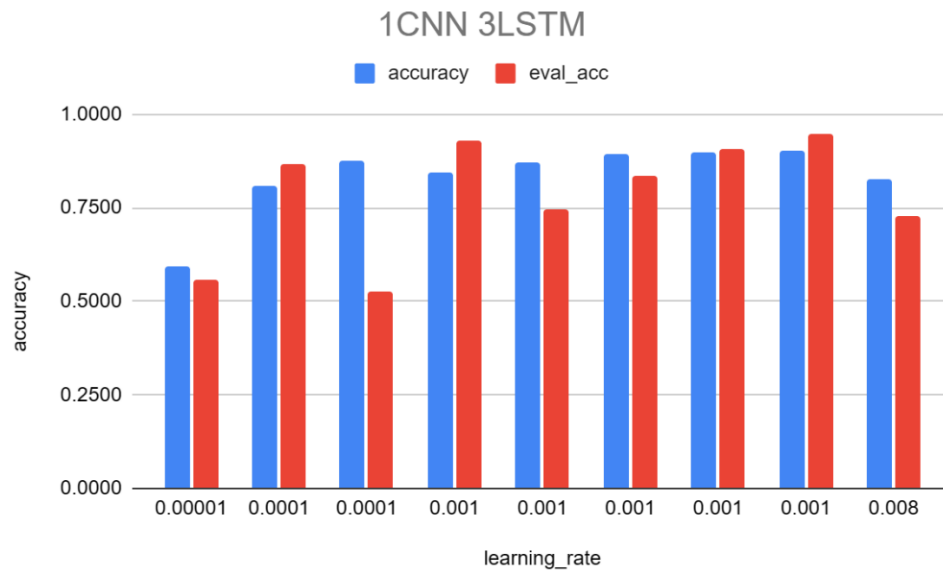
Tabel 4.1.2 Hasil Uji tanpa K-Fold

Layer	Learning rate	Neuron	Akurasi
CNN(2), LSTM(2)	0.001	256, 128 & 128, 128	0.8706
		256, 128 & 128, 128	0.8600
		256, 128 & 128, 128	0.8549
		256, 128 & 128, 128	0.8459
		256, 128 & 128, 128	0.7440
		256, 128 & 128, 128	0.7422
	0.000001	256, 128 & 128, 128	0.7405
CNN(1), LSTM(2)	0.001	256 & 128, 128	0.8647
		256 & 128, 128	0.8602
		256 & 128, 128	0.8513
		256 & 128, 128	0.4966

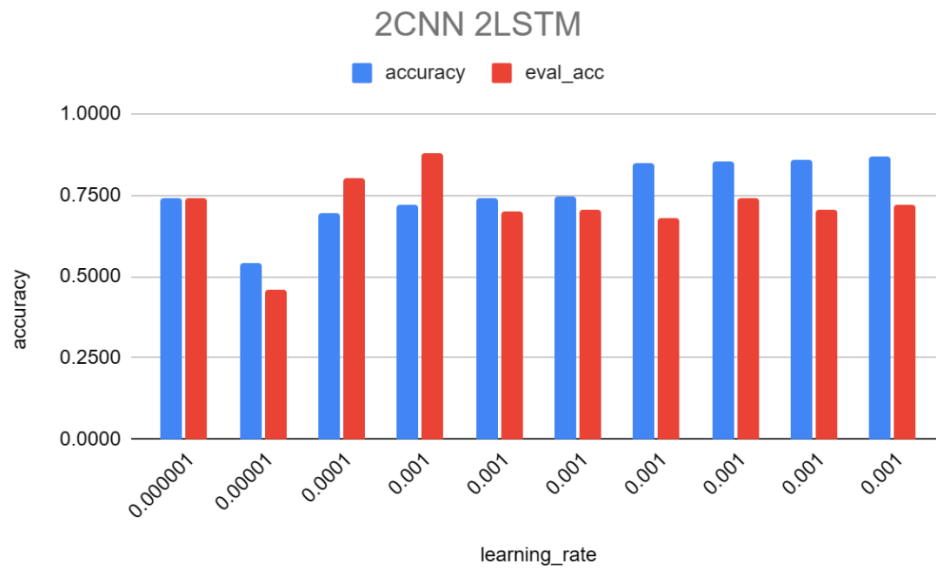
CNN(1), LSTM(3)	0.001	256 & 256, 128, 128	0.9005
		256 & 256, 128, 128	0.8965
		256 & 256, 128, 128	0.8937
		256 & 128, 128, 128	0.8695
	0.0001	256 & 256, 128, 128	0.8736
	0.008	256 & 256, 128, 128	0.8269
CNN(4), LSTM(0)	0.001	256, 128, 128, 128	0.3000
	0.0001	32, 32, 128, 64	0.2943
	0.00001	64, 128, 256, 32	0.2882
CNN(0), LSTM(4)	0.001	256, 128, 128, 128	0.8706
	0.0001	64, 512, 512, 32	0.8071
	0.00001	512, 256, 512, 128	0.5515
	0.001	128, 128, 64, 32	0.9048
CNN(3), LSTM(1)	0.001	256, 128, 128 & 128	0.7473
	0.0001	64, 256, 128 & 256	0.3929
	0.00001	256, 512, 128 & 32	0.3722

4.2 Analisa Hasil Pengujian

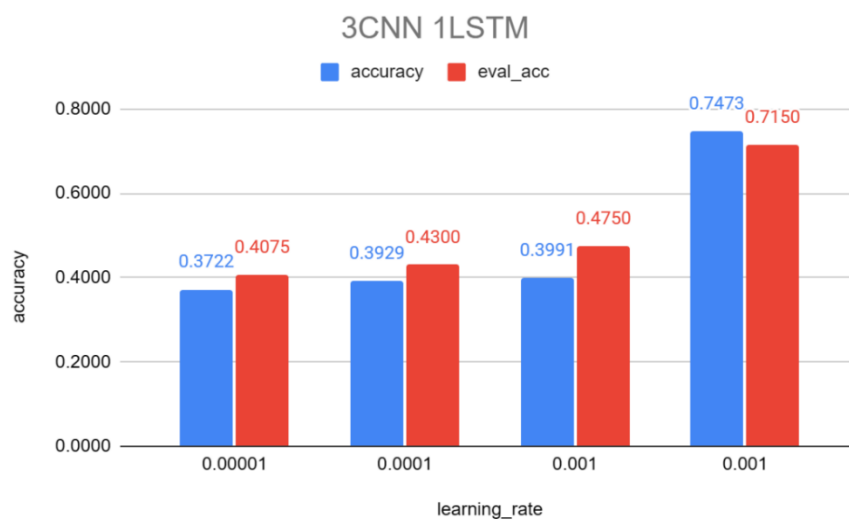
4.2.1 Pengujian Tanpa K-Fold



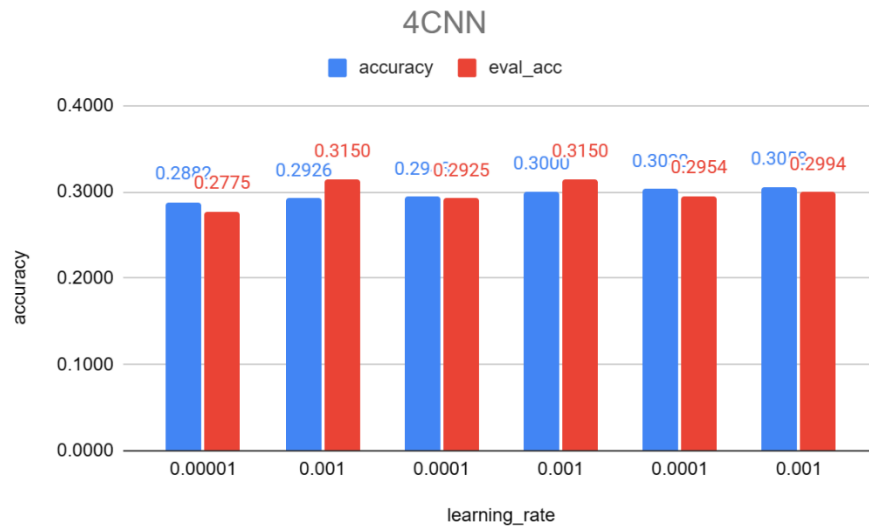
Gambar 4.2.1.1 Grafik Akurasi Arsitektur 3LSTM 1CNN



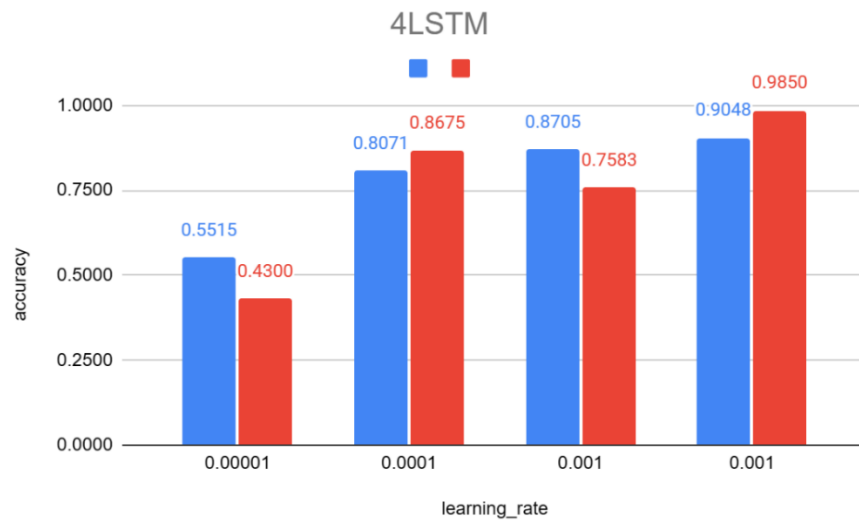
Gambar 4.2.1.2 Grafik Akurasi Arsitektur 2LSTM 2CNN



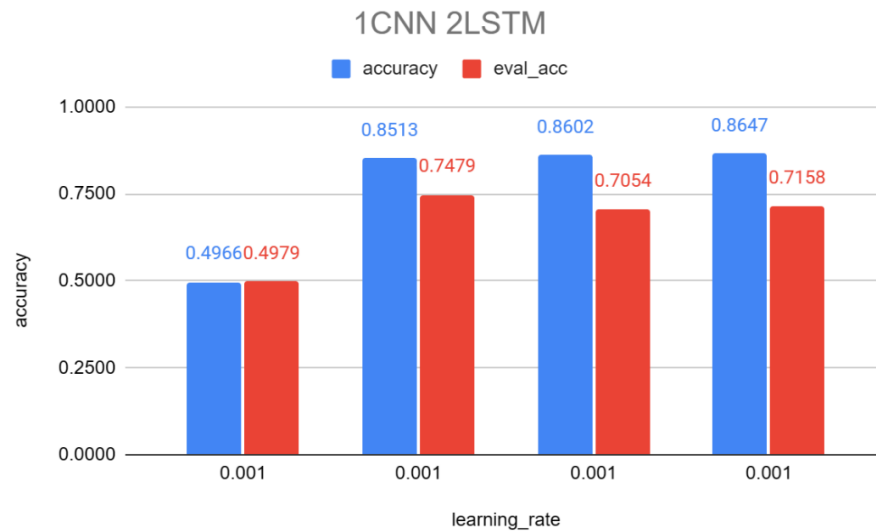
Gambar 4.2.1.3 Grafik Akurasi Arsitektur 1LSTM 3CNN



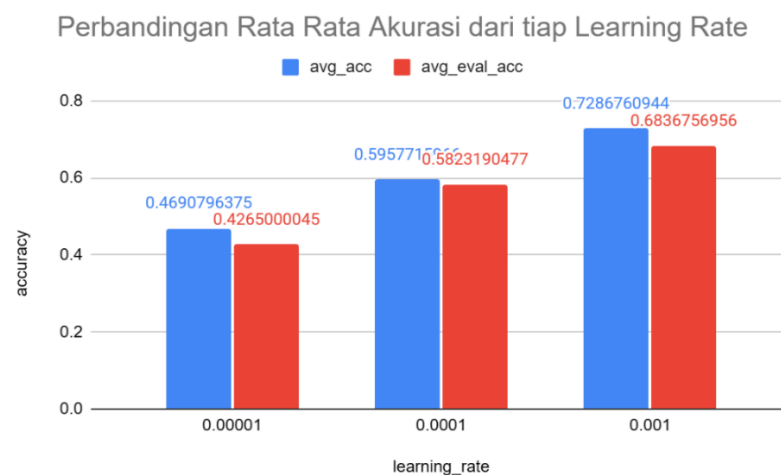
Gambar 4.2.1.4 Grafik Akurasi Arsitektur 4CNN



Gambar 4.2.1.5 Grafik Akurasi Arsitektur 4LSTM



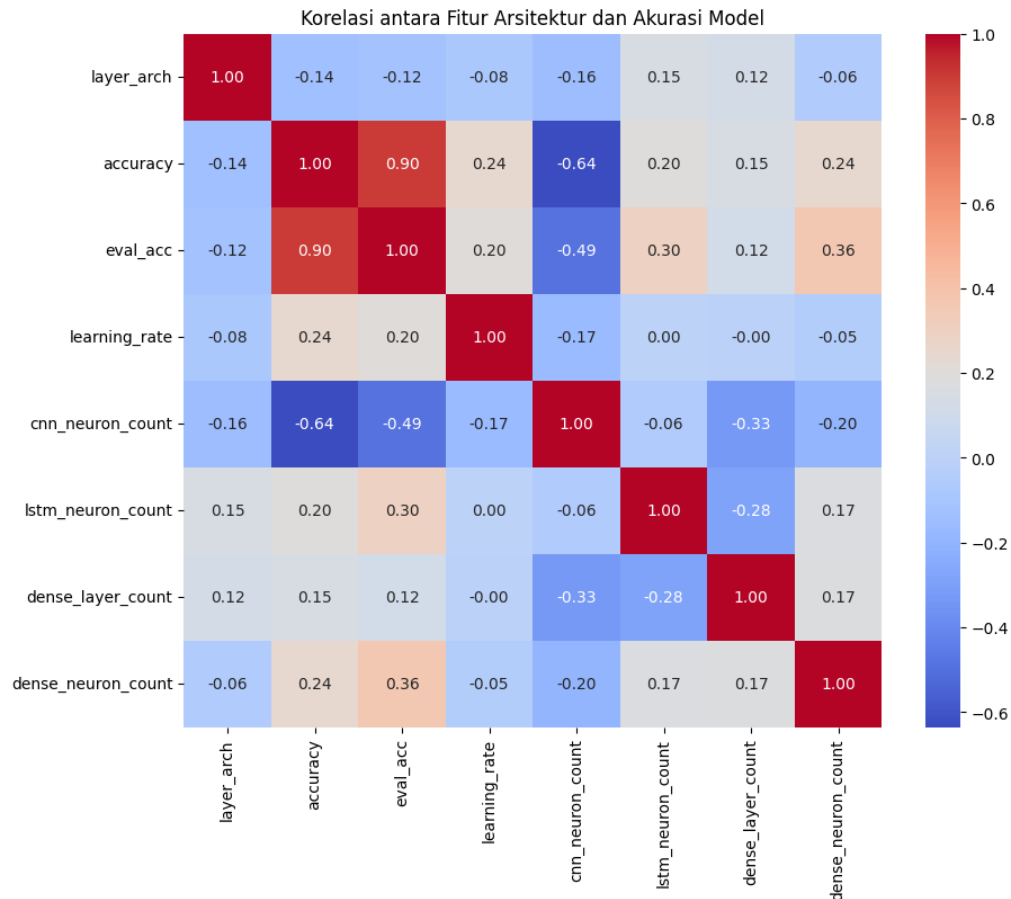
Gambar 4.2.1.6 Grafik Akurasi Arsitektur 2LSTM 1CNN



Gambar 4.2.1.7 Grafik Rata Rata Akurasi Learning Rate

Pada pengujian pertama, peneliti mengevaluasi setiap kemungkinan skema pengujian untuk menentukan model yang optimal sebelum melakukan pengujian menggunakan metode K-Folding. Berdasarkan analisis terhadap enam grafik yang disajikan, dapat disimpulkan bahwa arsitektur dengan empat lapisan LSTM menghasilkan akurasi tertinggi, mencapai 0.9048 , serta akurasi pada data baru sebesar 0.9850 . Learning rate yang secara konsisten menghasilkan akurasi tinggi

adalah $1-e3$ (0.001), dengan rata-rata akurasi 0.7363 dan rata-rata akurasi pada data baru 0.6967.



Gambar 4.2.1.8 Grafik Heatmap Korelasi Tanpa K-Fold

Dalam matriks korelasi diatas, aspek yang paling mencolok adalah nilai korelasi antara *cnn_neuron_count* (jumlah lapisan CNN beserta neuronnya) dengan *accuracy* (nilai akurasi pelatihan), yang tercatat sebesar -0.64 . Hal ini menandakan adanya hubungan korelasi negatif antara kedua variabel tersebut, yang mengimplikasikan bahwa peningkatan jumlah lapisan CNN berpotensi menurunkan nilai akurasi. Oleh karena itu, untuk mencapai akurasi pelatihan yang lebih tinggi, diperlukan konfigurasi dengan jumlah lapisan CNN yang lebih sedikit.

Dalam kontras dengan nilai *cnn_neuron_count*, nilai dari *lstm_neuron_count* dan *dense_neuron_count* menunjukkan kecenderungan positif. Meskipun tidak signifikan, hal ini mengindikasikan bahwa peningkatan jumlah lapisan LSTM dan dense dapat berkontribusi pada peningkatan akurasi.

4.2.2 Pengujian Dengan K-Fold

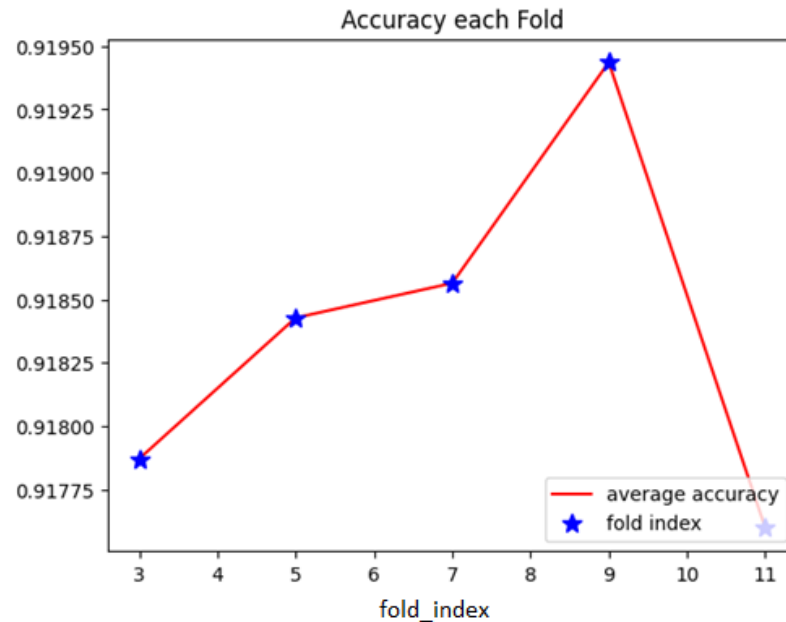
```
Best val_accuracy So Far: 0.9143134355545044
Total elapsed time: 00h 41m 28s
```

Gambar 4.2.2.1 Akurasi Data Baru dengan K-Fold

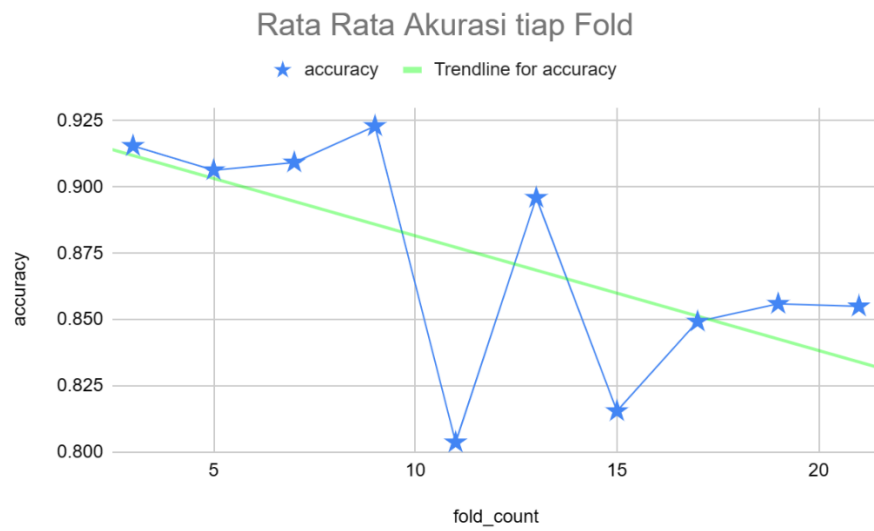
```
Fold ke 9 validation Loss: 0.207, Accuracy: 0.920,
epoch: 8, time_elapse: 5.378
```

```
-----
-
Average 9-fold accuracy: 0.9238836103015475
-----
```

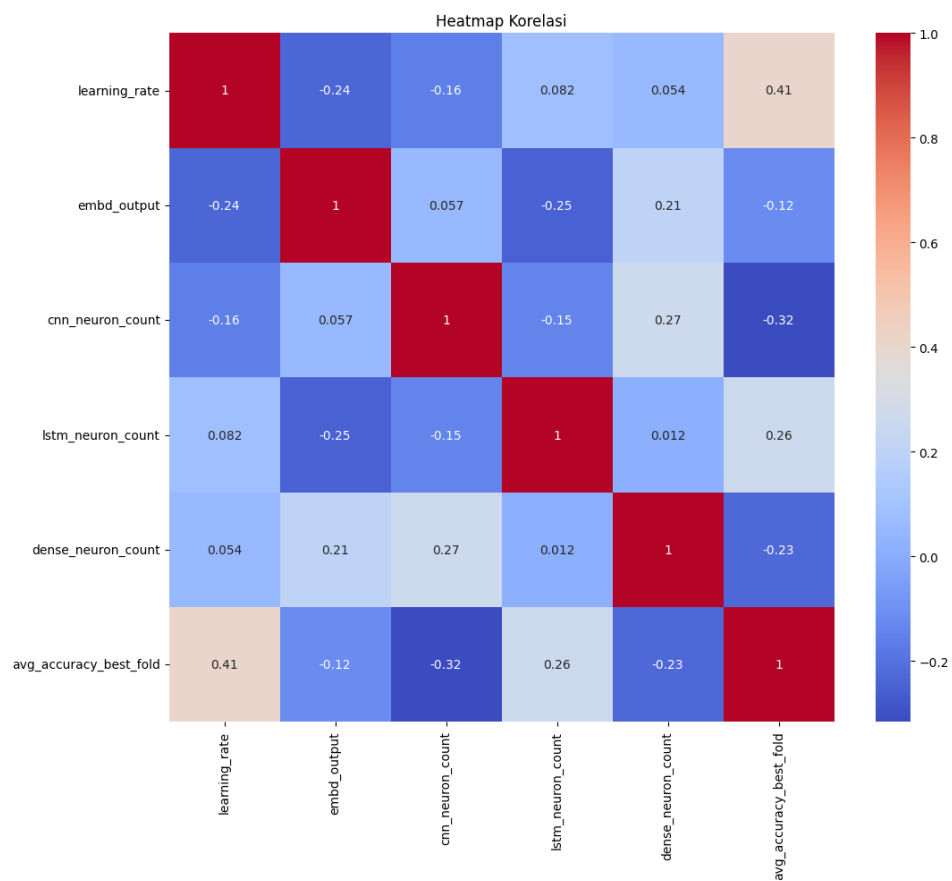
Gambar 4.2.2.2 Akurasi Data Latih dengan K-Fold



Gambar 4.2.2.3 Pengujian Pertama Crash



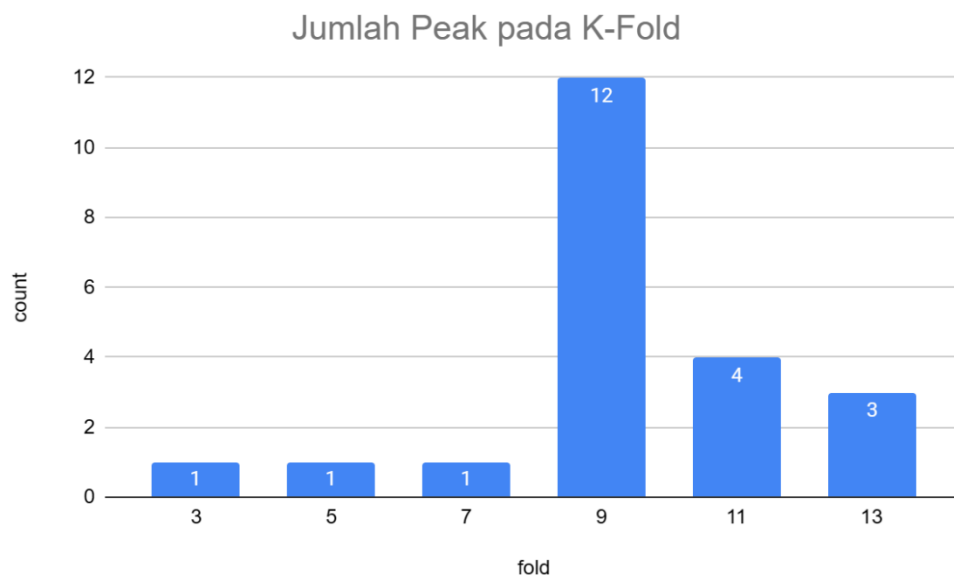
Gambar 4.2.2.4 Pengujian dengan Melakukan Fold Satu Persatu



Gambar 4.2.2.5 Grafik Heatmap Korelasi K-Fold

Dalam matriks korelasi di atas yang menggunakan validasi silang *K-Fold* sebagai penentu akurasi, terlihat adanya pola interaksi variabel yang serupa dengan matriks korelasi tanpa *K-Folding*; namun dengan nilai *learning_rate* yang lebih tinggi.

Penggunaan K-Fold ditujukan untuk mendapatkan stabilitas performa antara pengujian data baru dengan data latih, dan terbukti di Gambar 4.2.2.1 dan Gambar 4.2.2.2.



Gambar 4.2.2.6 Grafik Persebaran Puncak Akurasi Tertinggi

Pengujian pada **Gambar 4.2.2.3** dilakukan dengan sekali menjalankan program dan mengalami crash di fold ke 13. Pada gambar tersebut dapat dilihat bahwa puncak akurasi tertinggi ada pada fold ke-9. Begitu pula pada **Gambar 4.2.2.4**, dimana program dijalankan sekali tiap foldnya, menunjukkan puncak tertinggi pada fold ke-9.

Akurasi tertinggi pada pengujian menggunakan K-Fold adalah **0.92990** dengan arsitektur:

- 64 Neuron Emmbeding layer
- 4-layer LSTM dengan neuron [128, 128, 128, 128]
- Satu layer dense dengan neuron 32
- learning rate 0.001 (1-e3)

Arsitektur ini berhasil dijalankan dengan waktu **119.45** menit pada fold dengan akurasi tertinggi di **11-fold**.

Kemudian, jika melihat dari waktu latih rendah dengan akurasi tinggi, didapatkan arsitektur:

- 128 Neuron Emmbeding layer
- 3-layer LSTM dengan neuron [128, 128, 128]
- Satu layer CNN dengan neuron 128
- Satu layer Dense dengan neuron 16
- learning rate 0.001 (1-e3)

Arsitektur ini berhasil dijalankan dengan waktu **30.86** menit pada fold dengan akurasi tertinggi **0.92302** di **9-fold**.

BAB V

KESIMPULAN DAN SARAN

5.1. Kesimpulan

Dengan memanfaatkan sampel data dari Kaggle sebagai sumber data, Penelitian ini dilakukan sebagai respons terhadap maraknya pencurian akun YouTube akibat url phishing yang terjadi antara bulan Januari hingga Mei 2023. Data yang diambil dari Kaggle mencakup set data yang dimiliki oleh Manu Siddhartha pada tahun 2021, ditambah data white-list dari Phishing Database milik mitchellkroga pada github. Dalam percobaan pertama, peneliti melakukan eksperimen tanpa menggunakan metode K-Fold dengan tujuan untuk menemukan arsitektur model terbaik. Pada percobaan kedua, peneliti menggunakan metode K-Fold dalam satu run dan satu run per setiap Fold. Hasil dari penelitian ini adalah:

1. Komposisi layer LSTM penuh dapat menghasilkan akurasi tertinggi tetapi dengan bayaran komputasi yang lebih lama. Sedikit layer CNN (4 layer total dengan 1CNN dan sisanya LSTM) akan mempercepat komputasi secara signifikan dengan bayaran akurasi yang sedikit lebih rendah. Selain dua komposisi diatas, akan menghasilkan akurasi yang rendah bahkan sangat rendah.
2. Hyper parameter yang paling berdampak pada akurasi klasifikasi adalah jumlah layer CNN, jika terlalu banyak jumlah layernya maka akan menurunkan akurasi secara drastis, kemudian learning rate,

jika terlalu lambat maka proses belajar dari model akan terhambat diangka yang rendah.

Berdasarkan temuan penelitian ini, peneliti menyimpulkan bahwa penggabungan arsitektur Convolutional Neural Network (CNN) dengan Long Short-Term Memory (LSTM) secara efektif dapat digunakan untuk kasus klasifikasi. Terutama, dalam konteks pengklasifikasian URL, pendekatan ini menunjukkan efisiensi waktu dan akurasi yang baik.

5.2. Saran

Berdasarkan penelitian yang sudah dilaksanakan oleh peneliti, didapatkan beberapa saran dari hasil penelitian ini antara lain:

1. Penelitian dapat dilakukan di komputer dengan spesifikasi yang lebih mumpuni.
2. Mencoba menggunakan transfer learning agar mendapatkan akurasi yang lebih tinggi.
3. Mencoba menggunakan model tokenizer yang lebih advance seperti LLM, LLAMA atau GPT-Tokenizer.

DAFTAR PUSTAKA

- [1] H. huan Wang, L. Yu, S. wei Tian, Y. fang Peng, and X. jun Pei, "Bidirectional LSTM Malicious webpages detection algorithm based on convolutional neural network and independent recurrent neural network," *Appl. Intell.*, vol. 49, no. 8, pp. 3016–3026, Aug. 2019, doi: 10.1007/s10489-019-01433-4.
- [2] W. Wei, Q. Ke, J. Nowak, M. Korytkowski, R. Scherer, and M. Woźniak, "Accurate and fast URL phishing detector: A convolutional neural network approach," *Comput. Netw.*, vol. 178, Sep. 2020, doi: 10.1016/j.comnet.2020.107275.
- [3] A. Crisan, G. Florea, L. Halasz, C. Lemnaru, and C. Oprisa, "Detecting Malicious URLs Based on Machine Learning Algorithms and Word Embeddings," in *Proceedings - 2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing, ICCP 2020*, Institute of Electrical and Electronics Engineers Inc., Sep. 2020, pp. 187–193. doi: 10.1109/ICCP51029.2020.9266139.
- [4] P. Wanda and H. J. Jie, "URLDeep: Continuous Prediction of Malicious URL with Dynamic Deep Learning in Social Networks," *Int. J. Netw. Secur.*, vol. 21, pp. 971–978, Nov. 2019, doi: 10.6633/IJNS.846.
- [5] Z. Wang, X. Ren, S. Li, B. Wang, J. Zhang, and T. Yang, "A Malicious URL Detection Model Based on Convolutional Neural Network," *Secur. Commun. Netw.*, vol. 2021, 2021, doi: 10.1155/2021/5518528.
- [6] D. Harvey M, D. Paul J, and R. Nieto T, *Internet and World Wide Web: How to Program*. Prentice Hall, 2001. [Online]. Available: www.theadmin.data.bg
- [7] Giancarlo. Zaccone and Md. Rezaul. Karim, *Deep Learning with TensorFlow : Explore neural networks and build intelligent systems with Python, 2nd Edition*. Packt Publishing, 2018.
- [8] T. Oliver, *Machine Learning for Absolute Beginners: A Plain English Introduction*. Scatterplot Press, 2020.
- [9] G. Hirst and Y. Goldberg, *Neural Network Methods for Natural Language Processing*. Morgan & Claypool, 2017. [Online]. Available: <http://store.morganclaypool.com>
- [10] Alexandra George, *Python Text Mining: Perform Text Processing, Word Embedding, Text Classification and Machine Translation*. BPB Publications, 2022.
- [11] H. Hafid, "Penerapan K-Fold Cross Validation untuk Menganalisis Kinerja Algoritma K-Nearest Neighbor pada Data Kasus Covid-19 di Indonesia," 2023.
- [12] G. Aurélien, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, 2019.

LAMPIRAN

Lampiran 1 Tahapan Pengambilan Data

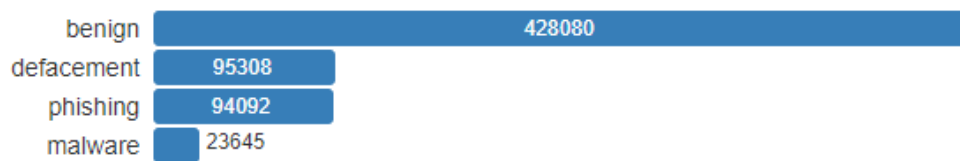
Dalam penelitian ini, dataset URL diambil dari Kaggle dan ditambah dengan data white list dari Phising Database milik mitchellkrogza. Data utama adalah data dari Kaggle karena data tersebut sudah dikurasi dari beberapa sumber data kolektif dari Perusahaan (Canadian Institute for Cybersecurity, PishTank, PishStrom), sedangkan dataset dari mitchellkrogza divalidasi oleh tim mereka, sehingga data dari Kaggle dirasa lebih reliable untuk digunakan sebagai data inti. Berikut adalah data dari Kaggle dan whitelist dari mitchellkrogza:

	url	type
0	100realt.ru	benign
1	104.219.234.142	benign
2	1040taxbiz.com	benign
3	1drv.ms	benign
4	213.238.34.213	benign
...	...	...
597	docs.google.com	benign
598	drive.google.com	benign
599	firebasestorage.googleapis.com	benign
600	googleusercontent.com	benign
601	onedrive.live.com	benign

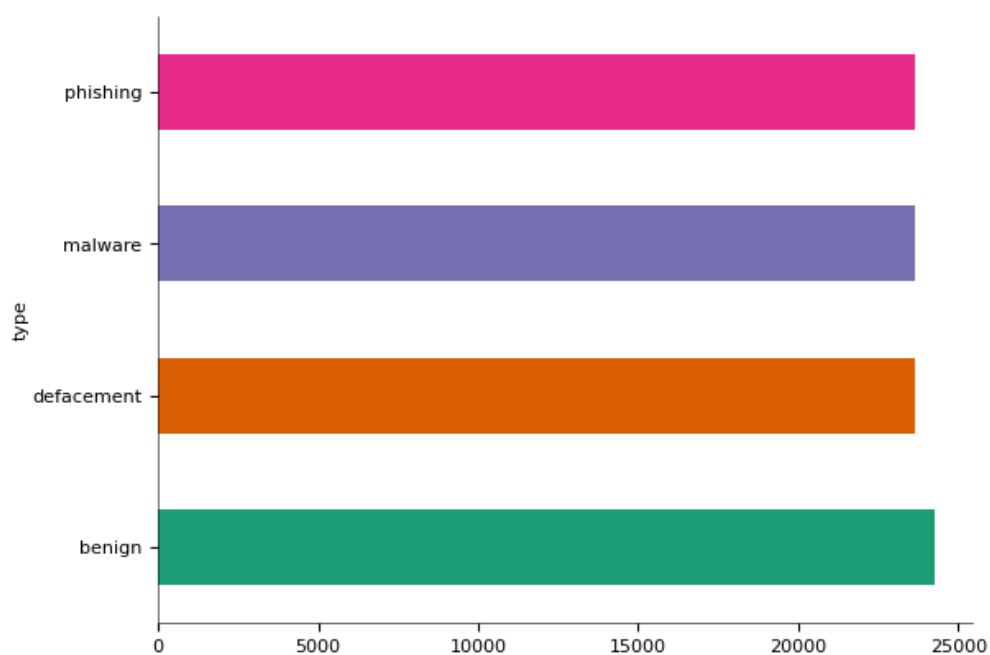
	url	type
0	ftp://188.128.111.33/IPTV/TV1324/view.html	phishing
1	ftp://188.128.111.33/web/sec.htm	phishing
2	ftp://me@createkindlebooks.org:Noobasshole@cre...	phishing
3	http://000000000000000000000000000000000000...	phishing
4	http://000000000000000000000000000000000000d...	phishing
...	...	...
1040817	http://zzzxm.mylp.kz/?new=1626225230407	phishing
1040818	http://zzzxxchatwhatsappcrakdhv.duckdns.org	phishing
1040819	http://zzzzascz.000webhostapp.com	phishing
1040820	http://zzzzmetamaskzzzz.dray-dns.de/recover	phishing
1040821	http://zzzzz66666-nbdghvhgjs-mndghsvhgsj.s3.am...	phishing
1040822 rows × 2 columns		

Whitelists yang digunakan adalah whitelist di tanggal 10 Agustus 2023 di jam 18.00, dan data dari Kaggle adalah data versi terakhir dari repository-nya.

Lampiran 2 Tahapan Balancing



Tahapan preprocessing pertama yang diberlakukan pada data adalah balancing data, dikarena data yang didapat tidak seimbang maka dilakukan penyeimbangan data menggunakan metode under-sampling dengan mengambil sampel sebanyak data paling sedikit yaitu sebanyak 23645 data tiap kelas, sebelum diberlakukan under-sampling dilakukan drop data duplikat terlebih dahulu. Sehingga menghasilkan data balance dengan sebaran data sebagai berikut:



Lampiran 3 Tahapan Label Encoding

Proses encoding label dilakukan dengan cara mencatat berapa banyak nilai unik dari sebuah kolom, kemudian melakukan enumerasi untuk membuat dictionary secara otomatis. Hasilnya adalah sebagai berikut: ‘{'benign': 0, 'defacement': 1, 'phishing': 2, 'malware': 3}’.

Setelah dictionary terbentuk, proses mapping dilakukan pada kolom target untuk mengubah string label menjadi numerical label. Berikut adalah kode yang digunakan:

```
# ubah label jadi angka
def labeling_data(data):
    mapping_counters = {}
    value_counts = data['type'].value_counts()
    category_mapping = {category: int(idx) for idx, category in enumerate(value_counts.index)}
    mapping_counters['type'] = category_mapping

    data['type'] = data['type'].map(category_mapping)
    return category_mapping
```

Lampiran 4 Tahapan Tokenisasi

Tokenisasi dilakukan dengan memanfaatkan paket modul standar python urllib.parse. Langkah pertama yang dilakukan adalah dengan melakukan parsing atau memecah URL berdasarkan komponen komponennya, yaitu skema, domain, path, query.

```
def tokenize_url(url):
    # Parse the URL
    parsed_url = urlparse(url)
```

Selanjutnya, mengambil data komponen yang sudah diparsing diambil dan setiap kata di dalamnya diekstrak menggunakan regex, lalu dimasukan ke dalam sebuah array.

```
# Extract components
scheme = parsed_url.scheme
domain = parsed_url.netloc
path = parsed_url.path

# Tokenize the scheme, domain, and path
scheme_tokens = re.findall(r'\w+', scheme) if scheme else []
domain_tokens = re.findall(r'\w+', domain) if domain else []
path_tokens = re.findall(r'\w+', path) if path else []
```

Proses serupa juga dilakukan pada komponen query, di mana ekspresi reguler digunakan untuk mengekstrak kata-kata dan memasukkannya ke dalam array.

```
# Tokenize query parameters
query_params = parse_qs(parsed_url.query)
query_tokens = []
for key, values in query_params.items():
    query_tokens.extend(re.findall(r'\w+', key))
    for value in values:
        query_tokens.extend(re.findall(r'\w+', value))
```

Setelah semua proses yang telah dijelaskan selesai, data dari skema, domain, path dan query yang telah ditokenisasi digabungkan menjadi satu kedalam sebuah array. Array hasil penggabungan tersebut selanjutnya dikembalikan sebagai nilai balik dari fungsi “tokenize_url”.

Berikut ini merupakan ilustrasi penggunaan fungsi tokenize_url dengan menggunakan URL berikut sebagai contoh:

‘https://pages.store.office.com/addinsinstallpage.aspx?assetid=WA104382081&rs=en-US&correlationId=88d8fa8c-600f-36f3-eb63-8be5e9081364.’

Pertama, pemecahan URL ke masing masing bagian. Hasil pemecahan URL adalah sebagai berikut:

- Skema: ‘https’

- Domain: 'pages.store.office.com'
- Path: '/addinsinstallpage.aspx'
- Query: 'assetid=WA104382081&rs=en-US&correlationId=88d8fa8c-600f-36f3-eb63-8be5e9081364'

Kemudian, skema, domain, dan path ditokenisasi. Hasil tokenisasi adalah sebagai berikut:

- Token skema: ['https']
- Token domain: ['pages', 'store', 'office', 'com']
- Token path: ['addinsinstallpage', 'aspx']

Selanjutnya, parameter query ditokenisasi. Hasil tokenisasi parameter query adalah sebagai berikut:

- Token query: ['assetid', 'WA104382081', 'rs', 'en', 'US', 'correlationId', '88d8fa8c', '600f', '36f3', 'eb63', '8be5e9081364']

Menghasilkan:

```

Tokens for URL (erickson.edu/) 1:
['erickson', 'edu']

Tokens for URL (crosstitch.com/search?f=lib01) 2:
['crosstitch', 'com', 'search', 'f', 'lib01']

Tokens for URL (http://jalopnik.com/apple-wants-to-make-navigation-like-a-friend-reading-yo-1700721307/layer/) 3:
['http', 'jalopnik', 'com', 'apple', 'wants', 'to', 'make', 'navigation', 'like', 'a', 'friend', 'reading', 'yo',

Tokens for URL (hpieurope.com/kit-info.php?partNo=102852&lang=en) 4:
['hpieurope', 'com', 'kit', 'info', 'php', 'partno', '102852', 'lang', 'en']

Tokens for URL (youtube.com/watch?v=4nxZfgnGHa0) 5:
['youtube', 'com', 'watch', 'v', '4nxzfgnGHa0']

Tokens for URL (projects.latimes.com/hollywood/star-walk/jo-van-fleet/) 6:
['projects', 'latimes', 'com', 'hollywood', 'star', 'walk', 'jo', 'van', 'fleet']

Tokens for URL (encyclopedia2.thefreedictionary.com/Iraq+war+(disambiguation))) 7:
['encyclopedia2', 'thefreedictionary', 'com', 'iraq', 'war', 'disambiguation']

```

Kemudian berdasarkan list diatas, dalam tiap kalimat yang unik akan diwakilkan dengan angka yang unik, jika sama maka akan diwakilkan dengan angka yang sama.

```

# Tokenize the URLs
tokenizer = Tokenizer()
tokenizer.fit_on_texts(tokenized_urls)
sequences = tokenizer.texts_to_sequences(tokenized_urls)

# Pad the sequences to a fixed length
X_padded = pad_sequences(sequences, maxlen=max_sequence_length)

y_encoded = to_categorical(labels)

word_index = tokenizer.word_index
print('Found %s unique tokens.' % len(word_index))

Found 116268 unique tokens.

```

Pada data peneliti terdapat 116.268 ribu token untuk dilatih.

Lampiran 5 Tahapan Cleaning

Proses pembersihan data pertama yang diterapkan adalah mengubah semua huruf dalam data menjadi huruf kecil. Langkah ini penting untuk menghindari bias antara huruf besar dan kecil saat melakukan pelatihan model. Selanjutnya, simbol-simbol umum dalam URL seperti titik dua (:), garis miring (/), titik (.), koma (,), dan simbol lain yang termasuk dalam tanda baca dihilangkan. Berikut adalah implementasi kode untuk melakukan dua proses tersebut:

```
# Convert to lowercase
token = token.lower()
# Remove punctuation and special characters
token = ''.join(char for char in token if char not in string.punctuation)
cleaned_tokens.append(token)
```

Dalam implementasi ini, penulis memanfaatkan daftar punctuation yang telah disediakan oleh modul string untuk menghilangkan karakter yang termasuk dalam tanda baca dari setiap string yang telah ditokenisasi.

```
1 import string
2 result = string.punctuation
3 print(result)

!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Lampiran 6 K-Fold Cross Validation

Evaluasi performa model komputasi dilakukan dengan teknik Cross Validation K-Fold, di mana parameter K ditetapkan dengan nilai minimum 3 dan maksimum 15. Proses iterasi akan berlangsung secara berkelanjutan hingga ditemukan kesalahan (error) atau tercapainya batas maksimum K yang telah ditentukan.

```
# Iterate over different fold values
def train_using_fold(fold_values = 3, threshold = 15):
    while True:
        if fold_values > threshold:
            break
```

```
for index, val_index in enumerate(kf.split(X_padded, y_single)):
    X_train_fold, X_val_fold = X_padded[val_index[0]], X_padded[val_index[1]]
    y_train_fold, y_val_fold = y_encoded[val_index[0]], y_encoded[val_index[1]]

    model = tuner.hypermodel.build(best_hps)
```

```
-----
Training with 11-fold cross-validation
-----
```

```
Fold ke 1 validation_loss: 0.216, accuracy: 0.904, epoch: 6, time_elapse: 4.541 mnt
Fold ke 2 validation_loss: 0.234, accuracy: 0.910, epoch: 7, time_elapse: 5.223 mnt
Fold ke 3 validation_loss: 0.223, accuracy: 0.911, epoch: 7, time_elapse: 5.254 mnt
Fold ke 4 validation_loss: 0.243, accuracy: 0.900, epoch: 6, time_elapse: 4.518 mnt
Fold ke 5 validation_loss: 0.234, accuracy: 0.910, epoch: 6, time_elapse: 4.531 mnt
Fold ke 6 validation_loss: 0.223, accuracy: 0.908, epoch: 6, time_elapse: 4.557 mnt
Fold ke 7 validation_loss: 0.210, accuracy: 0.916, epoch: 7, time_elapse: 5.212 mnt
Fold ke 8 validation_loss: 0.228, accuracy: 0.908, epoch: 6, time_elapse: 4.553 mnt
Fold ke 9 validation_loss: 0.219, accuracy: 0.909, epoch: 6, time_elapse: 4.538 mnt
Fold ke 10 validation_loss: 0.251, accuracy: 0.910, epoch: 7, time_elapse: 5.199 mnt
An error occurred: Graph execution error
```

Lampiran 7 Tahapan Optimalisasi

```
delete_variables([
    'model', 'X_train_fold',
    'X_val_fold', 'y_train_fold',
    'y_val_fold', 'train_generator',
    'val_generator', 'end_time',
    'start_time', 'time_used'
])
```

Blok kode optimisasi RAM untuk mencegah terjadinya kegagalan sistem (*crash*) selama proses iterasi K-Fold. Blok kode ini beroperasi dengan mengeliminasi variabel-variabel yang tidak lagi diperlukan dan akan ditugaskan ulang pada iterasi berikutnya, sehingga meminimalisir penggunaan memori yang berlebihan dan meningkatkan efisiensi dalam pengelolaan sumber daya komputasi. Penggunaan cara ini menghasilkan penurunan penggunaan ram yang cukup signifikan.

Lampiran 8 Tabel Percobaan dengan penerapan optimalisasi

fold	iterasi	tanpa optimisasi	dengan optimisasi
5 Fold	iterasi 1	1342	1346
	iterasi 2	1518	2,388
	iterasi 3	1654	3,466
	iterasi 4	1816	4,520
	iterasi 5	1972	5,580
rata rata perubahan		78.75 mb	157.5 mb

Lampiran 9 Tabel Percobaan tanpa penerapa optimalisasi

fold	iterasi	optimisasi	Tanpa optimisasi
5 Fold	iterasi 1	1318	1340
	iterasi 2	1486	2,542
	iterasi 3	1622	3,766
	iterasi 4	1780	4976
	iterasi 5	1932	6,188
rata rata perubahan		153.5mb	1212mb

Lampiran 10 Proses dalam Layer Embedding

Pada layer ini, kumpulan data hasil tokenisasi diproses menjadi vektor embedding berdimensi tertentu, tergantung dengan nilai parameter output shapenya. Bobot vektor awal akan diinisialisasi secara acak dan akan diperbaharui selama proses pelatihan. Contoh ini siasi bobot vektor dan dimensi vektor sebagai berikut:

```
['https', 'pages', 'store', 'office', 'com', 'addinsinstallpage', 'aspx', 'assetid',
'WA104382081', 'rs', 'en', 'US', 'correlationId', '88d8fa8c', '600f', '36f3', 'eb63',
'8be5e9081364']
```

Gambar dibawah menggunakan output 2 dimensi, sehingga vector akan berdimensi dua.

Lampiran 11 Hasil Matriks Embedding layer dengan output 2

```
<tf.Tensor: shape=(1, 18, 2), dtype=float32, numpy=
array([[[-0.01908034,  0.03861847],
        [ 0.03995821, -0.03053806],
        [-0.04120221, -0.01564108],
        [-0.02079078,  0.01690531],
        [ 0.01385763, -0.03331822],
        [-0.01176133, -0.00384615],
        [ 0.00248452,  0.02336382],
        [ 0.02789558, -0.02883978],
        [ 0.03789785,  0.00427191],
        [-0.01920336,  0.02379079],
        [-0.02115351,  0.01275298],
        [ 0.0332779 ,  0.00946003],
        [ 0.03966541, -0.00348798],
        [-0.04169518, -0.01172306],
        [-0.04120693, -0.00742741],
        [ 0.04015583,  0.01533734],
        [-0.03361259,  0.01022542],
        [ 0.00825135, -0.01846467]]], dtype=float32)>
```

Gambar dibawah menggunakan output 4 dimensi, sehingga vector akan berdimensi empat.

Lampiran 12 Hasil Matriks Embedding layer dengan output 4

```
<tf.Tensor: shape=(1, 18, 4), dtype=float32, numpy=
array([[[ 0.02210157, -0.01442808,  0.02508466,  0.04504886],
        [-0.02520921, -0.03011799, -0.02329307, -0.04766263],
        [ 0.03661301,  0.022317  , -0.04105455,  0.04243096],
        [-0.01488104,  0.0186   ,  0.02535516,  0.00488672],
        [ 0.02387745,  0.01679016, -0.02364152,  0.0060959 ],
        [-0.00167899,  0.03132757,  0.03962976, -0.03286178],
        [-0.02054981, -0.00757687,  0.01114387, -0.01254107],
        [-0.02406098, -0.02980325,  0.04636644, -0.04492496],
        [ 0.02641353,  0.00453936, -0.0035538 ,  0.03738636],
        [ 0.01614766,  0.00017399,  0.03473053, -0.00820721],
        [ 0.01185832,  0.0079677 , -0.02548361,  0.01163862],
        [ 0.0138914 ,  0.04237558, -0.00754281, -0.0149905 ],
        [-0.00262392, -0.01472669, -0.0203277 ,  0.0456812 ],
        [-0.01813231,  0.0473469 ,  0.02314356, -0.02567519],
        [-0.011436  , -0.0248998 ,  0.01269251, -0.02137074],
        [-0.04887975, -0.03286751,  0.04081466,  0.00357535],
        [-0.0179581 ,  0.03012378, -0.00155519,  0.03977081],
        [-0.04195763,  0.03917981,  0.03344394,  0.00230271]]],
      dtype=float32)>
```

Lampiran 13 Proses dalam layer CNN

Pada layer CNN proses yang dilakukan adalah mengekstrak fitur penting yang relevan dengan data sekarang sehingga matriks yang didapat lebih informatif dibandingkan sebelum masuk ke CNN. Mirip seperti CNN pada citra, CNN pada teks melakukan proses ekstraksi fitur dalam dimensi satu. Rumus yang digunakan adalah:

$$F(x) = (I * W)(x) = \sum_{i=1}^m I(x+i) \cdot W(i)$$

(5.2.1)

Di mana:

- $F(x)$ adalah nilai pada feature map.
- $I(x+i)$ adalah vektor kata pada posisi $x+i$.
- $W(i)$ adalah bobot filter pada posisi i .

Contoh pengerjaan jika kita memiliki matriks vektor $I = [3,1,4,2,5]$, kemudian menerapkan CNN satu dimensi pada vektor tersebut dengan filter 2×1 : $g(x) = [\frac{1}{2}, \frac{1}{2}]$. maka menghasilkan:

$$h(1) = 3 * \frac{1}{2} + 1 * \frac{1}{2} = 2$$

$$h(2) = 1 * \frac{1}{2} + 4 * \frac{1}{2} = 2.5$$

$$h(3) = 4 * \frac{1}{2} + 2 * \frac{1}{2} = 3$$

$$h(4) = 2 * \frac{1}{2} + 5 * \frac{1}{2} = 3.5$$

Sehingga menghasilkan matriks vektor $I = [2, 2.5, 3, 3.5]$

Layer pooling adalah bagian dari CNN yang berfungsi untuk mengurangi dimensi data dan mengurutkan layer dalam CNN. Peneliti menggunakan jenis max pooling, dimana mengambil data maksimum untuk merepresentasikan satu area filter, sama de. Rumus yang digunakan adalah:

$$M(x) = \max (I(x + i)) \quad (5.2.2)$$

Di mana:

- $M(x)$ adalah nilai hasil max pooling pada posisi x .
- $I(x + i)$ adalah nilai vektor pada posisi $x + i$

Contoh pengerjaan, jika kita memiliki matriks vektor $I = [3,1,4,2,5]$, kemudian menerapkan maxpooling pada vektor tersebut maka menghasilkan:

$$M(1) = \max(3,1) = 3$$

$$M(2) = \max(1,4) = 4$$

$$M(3) = \max(4,2) = 4$$

$$M(4) = \max(2,5) = 5$$

Sehingga menghasilkan matriks vektor $I = [3, 4, 4, 5]$

Neuron yang divariasikan dalam program peneliti adalah adalah [32, 64, 128, 256, 512]

Lampiran 14 Hasil CNN dengan filter 4

```
<tf.Tensor: shape=(1, 8, 4), dtype=float32, numpy=
array([[ [0.03473107, 0.02187518, 0.01246977, 0.04695098],
        [0.01907397, 0.01732575, 0.04330425, 0.
        ],
        [0.0511762 , 0.01654365, 0.008924 , 0.03912671],
        [0.01434586, 0.02129882, 0.04663947, 0.
        ],
        [0.05372129, 0.
        , 0.07606247, 0.02438662],
        [0.00436814, 0.
        , 0.02430429, 0.04837126],
        [0.05776923, 0.02574828, 0.01425277, 0.05624728],
        [0.02372896, 0.
        , 0.00107041, 0.03720537]]], dtype=float32)>
```

Lampiran 15 Hasil CNN dengan filter 16

```
<tf.Tensor: shape=(1, 8, 16), dtype=float32, numpy=
array([[ [1.48926079e-02, 0.00000000e+00, 2.06639105e-03, 0.00000000e+00,
        2.94117667e-02, 5.08075627e-03, 3.33999395e-02, 5.45279216e-03,
        1.41677530e-02, 0.00000000e+00, 2.84600668e-02, 0.00000000e+00,
        2.91180015e-02, 1.23839602e-02, 1.17519179e-04, 2.32329983e-02],
        [0.00000000e+00, 1.29453484e-02, 1.39715029e-02, 1.60263479e-02,
        2.07714681e-02, 2.32968405e-02, 2.11847648e-02, 3.45731936e-02,
        8.16360291e-04, 1.49257695e-02, 8.69292580e-03, 0.00000000e+00,
        2.59065032e-02, 6.99666853e-04, 5.69671858e-03, 1.79409552e-02],
        [3.53544988e-02, 2.21422464e-02, 0.00000000e+00, 3.66051309e-02,
        7.22811883e-03, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
        2.50222045e-03, 0.00000000e+00, 5.18946210e-03, 3.71199064e-02,
        1.91558655e-02, 5.05618611e-03, 4.14379826e-03, 3.86455953e-02],
        [0.00000000e+00, 2.30575111e-02, 4.05312050e-03, 5.74954793e-05,
        1.36047434e-02, 2.18070652e-02, 0.00000000e+00, 2.27516959e-03,
        0.00000000e+00, 1.74139254e-02, 1.64490975e-02, 0.00000000e+00,
        4.21144031e-02, 1.60271919e-03, 1.94600057e-02, 2.85922270e-02],
        [7.81029137e-03, 0.00000000e+00, 0.00000000e+00, 1.03773950e-02,
        2.86619794e-02, 1.98782478e-02, 2.35219044e-03, 4.32034722e-03,
        0.00000000e+00, 3.18149813e-02, 2.63747498e-02, 0.00000000e+00,
        2.85614580e-02, 1.58753060e-02, 1.10180434e-02, 1.38303882e-03],
        [0.00000000e+00, 3.06435432e-02, 0.00000000e+00, 0.00000000e+00,
        4.13652658e-02, 2.92836092e-02, 3.01448777e-02, 0.00000000e+00,
        0.00000000e+00, 2.56829639e-03, 2.39640325e-02, 1.64783895e-02,
        2.83948388e-02, 3.14341523e-02, 2.00690771e-03, 0.00000000e+00],
        [0.00000000e+00, 0.00000000e+00, 1.32226506e-02, 2.13492252e-02,
        4.58994284e-02, 1.16915014e-02, 1.20351110e-02, 7.22733047e-03,
        2.75951233e-02, 1.82301016e-03, 2.36995928e-02, 0.00000000e+00,
        9.37517360e-03, 0.00000000e+00, 0.00000000e+00, 6.35125302e-03],
        [1.22744804e-02, 1.37753012e-02, 1.22582903e-02, 3.07210744e-03,
        0.00000000e+00, 1.44323343e-02, 2.17013806e-02, 1.27521679e-02,
        3.08081564e-02, 1.36113521e-02, 2.05645170e-02, 2.40124054e-02,
        0.00000000e+00, 9.28519946e-03, 1.32353911e-02, 9.23374668e-04]]],
dtype=float32)>
```

Lampiran 16 Proses dalam Layer LSTM

Long Short-Term Memory (LSTM) adalah versi lain dari RNN dengan tujuan untuk mengatasi masalah vanishing gradient pada RNN. LSTM memiliki unit memori internal yang disebut cell state, unit ini dapat menyimpan informasi dari waktu sebelumnya dan memutuskan apakah informasi tersebut akan diteruskan atau dilupakan, LSTM juga memiliki tiga gerbang (gates): input gate, forget gate, dan output gate, gerbang-gerbang ini mengatur aliran informasi dalam dan keluar dari cell state. Rumus rumus dari LSTM adalah sebagai berikut:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (5.2.3)$$

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5.2.4)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (5.2.5)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5.2.6)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (5.2.7)$$

Di mana i_t adalah Input Gate, f_t adalah Forget Gate, C_t adalah Update Cell State, o_t adalah Output Gate, dan h_t adalah Hidden State, kemudian:

- x_t adalah input pada waktu t .
- h_{t-1} adalah hidden state pada waktu sebelumnya.
- W_i, W_f, W_c, W_o adalah matriks bobot.
- b_i, b_f, b_c, b_o adalah bias.
- σ adalah fungsi sigmoid.

- *tanh* adalah fungsi tangen hiperbolik.

Layer LSTM mengenali pola data dari layer sebelumnya.

```
<tf.Tensor: shape=(1, 4), dtype=float32, numpy=
array([[ -0.01316059,  0.0015068 , -0.00633642, -0.0036962 ]],
      dtype=float32)>
```

Lampiran 17 Seluruh Data Pengujian tanpa K-Fold

layer_arch	accuracy	eval_acc	learning_rate	cnn_neuron_count	lstm_neuron_count	dense_layer_count	dense_neuron_count
CNN(0), LSTM(4)	0.9048	0.9850	0.001	N/A	128, 128, 64, 32	3	256, 32, 512
CNN(1), LSTM(3)	0.9005	0.9488	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8965	0.9065	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8937	0.8333	0.001	256	256, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8736	0.5258	0.0001	256	256, 128, 128	2	128, 64
CNN(2), LSTM(2)	0.8706	0.7208	0.001	256, 128	128, 128	2	128, 64
CNN(0), LSTM(4)	0.8705	0.7583	0.001	N/A	256, 128, 128, 128	2	128, 64
CNN(1), LSTM(3)	0.8695	0.7437	0.001	256	128, 128, 128	1	64
CNN(1), LSTM(2)	0.8647	0.7158	0.001	256	128, 128	3	128, 64, 32
CNN(1), LSTM(2)	0.8602	0.7054	0.001	256	128, 129	3	128, 64, 32
CNN(2), LSTM(2)	0.8600	0.7050	0.001	256, 128	128, 128	3	128, 64, 32
CNN(2), LSTM(2)	0.8549	0.7417	0.001	256, 128	128, 128	3	128, 64, 32
CNN(1), LSTM(2)	0.8513	0.7479	0.001	256	128, 128	3	128, 64, 32
CNN(1), LSTM(3)	0.8466	0.9300	0.001	64	128, 32, 32	2	512, 32
CNN(2), LSTM(2)	0.8459	0.6800	0.001	256, 128	128, 128	3	128, 64, 32
CNN(1), LSTM(3)	0.8269	0.7267	0.008	256	256, 128, 128	2	128, 64
CNN(0), LSTM(4)	0.8071	0.8675	0.0001	N/A	64, 512, 512, 32	2	512, 64
CNN(1), LSTM(3)	0.8070	0.8650	0.0001	256	128, 256, 64	2	128, 32
CNN(3), LSTM(1)	0.7473	0.7150	0.001	256, 128, 128	128	2	128, 64
CNN(2), LSTM(2)	0.7440	0.7050	0.001	256, 128	128, 128	2	128, 64
CNN(2), LSTM(2)	0.7422	0.7013	0.001	256, 128	128, 128	4	128, 64, 32, 16
CNN(2), LSTM(2)	0.7405	0.7405	0.000001	256, 128	128, 128	3	128, 64, 32
CNN(2), LSTM(2)	0.7183	0.8800	0.001	512, 128	512, 256	2	32, 32
CNN(2), LSTM(2)	0.6924	0.8000	0.0001	128, 128	512, 128	3	64, 512, 128
CNN(1), LSTM(3)	0.5927	0.5575	0.00001	512	64, 256, 512	1	128
CNN(0), LSTM(4)	0.5515	0.4300	0.00001	N/A	512, 256, 512, 128	3	128, 64, 256

CNN(2), LSTM(2)	0.5407	0.4600	0.00001	256, 512	128, 32	1	32
CNN(1), LSTM(2)	0.4966	0.4979	0.001	256	128, 128	3	128, 64, 32
CNN(3), LSTM(1)	0.3991	0.4750	0.001	32, 128, 256	32	2	32, 32
CNN(3), LSTM(1)	0.3929	0.4300	0.0001	64, 256, 128	256	3	128, 128, 512
CNN(3), LSTM(1)	0.3722	0.4075	0.00001	256, 512, 128	32	1	128
CNN(4), LSTM(0)	0.3058	0.2994	0.001	256, 128, 64, 32	N/A	2	128, 64
CNN(4), LSTM(0)	0.3030	0.2954	0.0001	256, 128, 64, 32	N/A	2	128, 64
CNN(4), LSTM(0)	0.3000	0.3150	0.001	256, 128, 128, 128	N/A	3	128, 64
CNN(4), LSTM(0)	0.2943	0.2925	0.0001	32, 32, 128, 64	N/A	3	32, 256, 256
CNN(4), LSTM(0)	0.2926	0.3150	0.001	64, 256, 32, 128	N/A	1	128
CNN(4), LSTM(0)	0.2882	0.2775	0.00001	64, 128, 256, 32	N/A	2	128, 256

Lampiran 18 Seluruh Data Pengujian Menggunakan K-Fold

fold_coun t	learn ing_r ate	embd _out put	layer _arc h	cn n_ ne u ro n_ c o u n t	lst m_ ne u ro n_ c o u n t	dense_ la yer_ cou n t	dense_ ne u ro n_ c o u n t	avg_ accu ra cy_ best_ fold	time_ elaps e_ best_ fold	avg_ epoc h_ best_ fold	K- Fold_ metho d	best_ fold
13	0.001	64	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	1	32	0.92990	119.45	6	Stratifi ed	11
11	0.001	256	CNN(0), LSTM(4)	N/A	128, 128, 256, 256	2	16, 64	0.92827	65.18	6	Stratifi ed	9
13	0.001	128	CNN(0), LSTM(4)	N/A	256, 128, 256, 128	2	64, 16	0.92718	58.00	7	Non- Stratifi ed	9
13	0.001	64	CNN(0), LSTM(4)	N/A	128, 256, 128, 256	1	128	0.92393	83.76	7	Stratifi ed	13
9	0.001	64	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	2	128, 64	0.92388	42.04	7	Stratifi ed	9
9	0.001	128	CNN(1), LSTM(3)	128	128, 128, 128	1	16	0.92302	30.86	6	Stratifi ed	9
11	0.001	64	CNN(1), LSTM(3)	256	128, 256, 128	2	64, 32	0.92108	48.15	6	Stratifi ed	9
11	0.001	128	CNN(1), LSTM(3)	128	256, 128, 128	1	32	0.91948	43.60	6	Stratifi ed	11
13	0.001	32	CNN(0), LSTM(4)	N/A	128, 128, 128, 128	2	64, 128	0.91928	59.34	7	Stratifi ed	13
3	0.001	32	CNN(0), LSTM(4)	N/A	128, 256, 128, 128	2	32, 64	0.91800	20.80	7	Stratifi ed	3
13	0.001	64	CNN(1), LSTM(3)	128	128, 128, 128	2	16, 32	0.91760	56.40	7	Stratifi ed	11
11	0.001	16	CNN(0), LSTM(4)	N/A	256, 256, 128, 128	2	64, 128	0.91724	109.83	7	Stratifi ed	9
9	0.001	64	CNN(1), LSTM(3)	256	128, 256, 256	1	16	0.91700	38.56	6	Stratifi ed	9
9	0.001	16	CNN(1), LSTM(3)	128	128, 64, 64	2	32, 32	0.91501	45.70	7	Non- Stratifi ed	9

11	0.001	64	CNN(1), LSTM(3)	128	128, 256, 256	1	64	0.91500	39.37	7	Stratified	9
9	0.001	16	CNN(1), LSTM(3)	128	128, 128, 128	1	32	0.91115	23.51	7	Stratified	7
15	0.001	16	CNN(1), LSTM(3)	128	128, 256, 256	2	128, 128	0.90714	58.27	7	Stratified	13
13	0.0001	128	CNN(1), LSTM(3)	256	128, 256, 128	2	64, 16	0.89967	70.48	7	Stratified	11
5	0.0001	256	CNN(1), LSTM(3)	128	128, 128, 128	2	32, 64	0.89115	17.37	7	Non-Stratified	5
11	0.0001	16	CNN(1), LSTM(3)	128	128, 256, 128	3	16, 16, 128	0.89054	58.42	8	Stratified	9
9	0.0001	64	CNN(0), LSTM(4)	N/A	128, 256, 256, 256	2	128, 64	0.88647	80.29	7	Non-Stratified	9
5	0.001	128	CNN(2), LSTM(2)	128, 64	128, 128	2	64, 32	0.82880	14.07	8	Non-Stratified	9

Lampiran 19 Arsitektur Model Terbaik Pengujian K-Fold

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 157, 64)	7441216
lstm (LSTM)	(None, 157, 128)	98816
lstm_1 (LSTM)	(None, 157, 128)	131584
lstm_2 (LSTM)	(None, 157, 128)	131584
lstm_3 (LSTM)	(None, 128)	131584
dense (Dense)	(None, 32)	4128
dense_1 (Dense)	(None, 4)	132

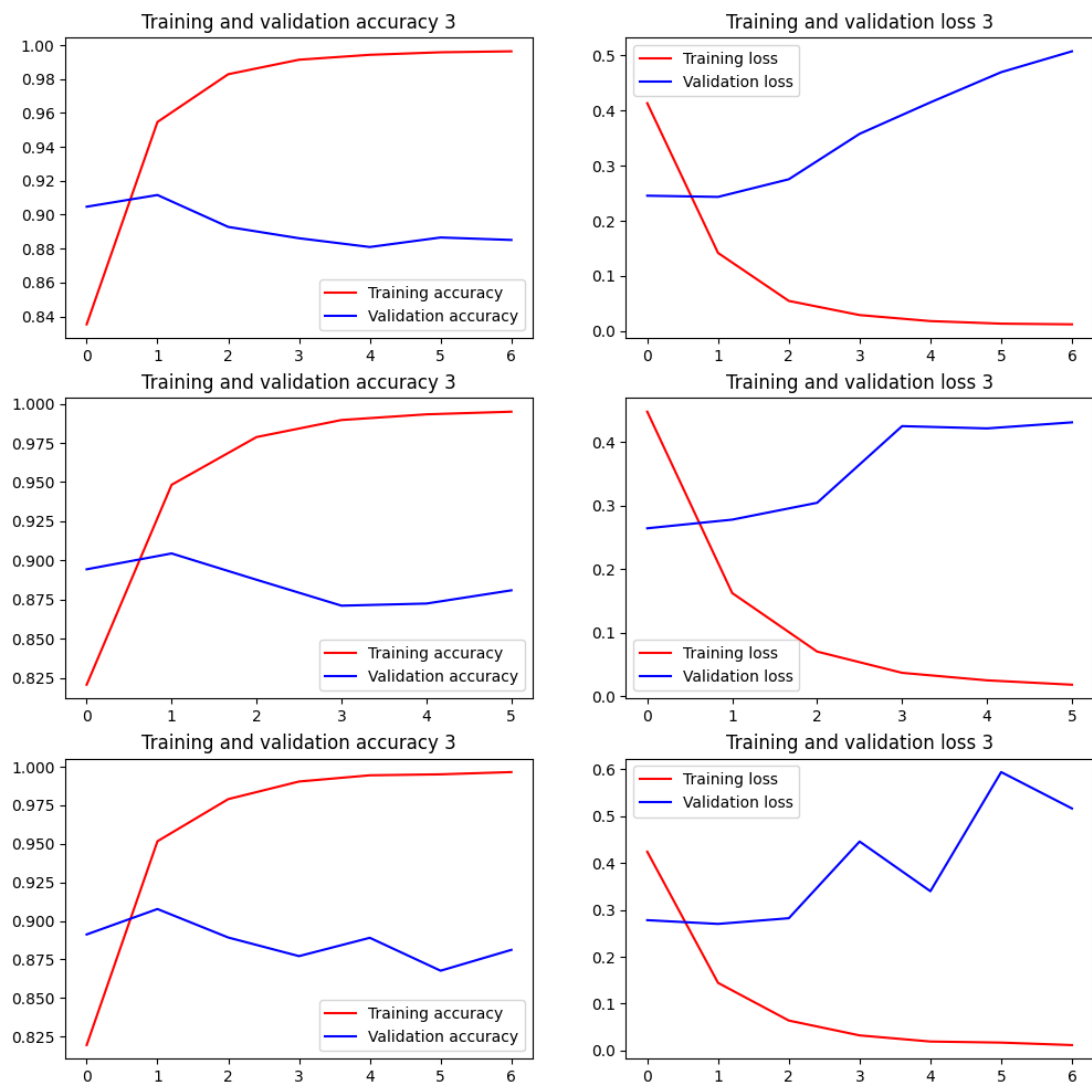
```

=====
Total params: 7939044 (30.29 MB)
Trainable params: 7939044 (30.29 MB)
Non-trainable params: 0 (0.00 Byte)

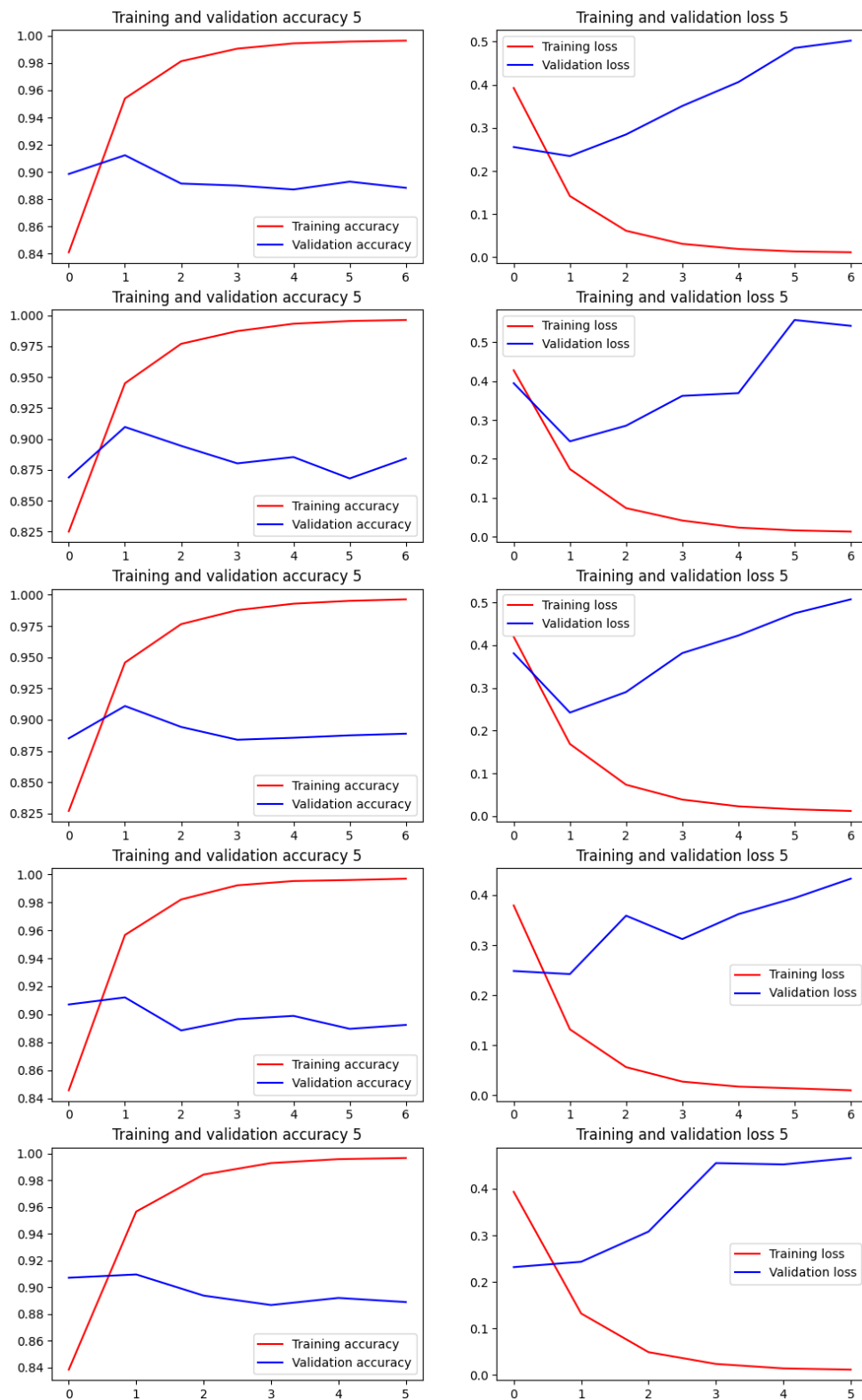
```

Learning Rate: 0.001

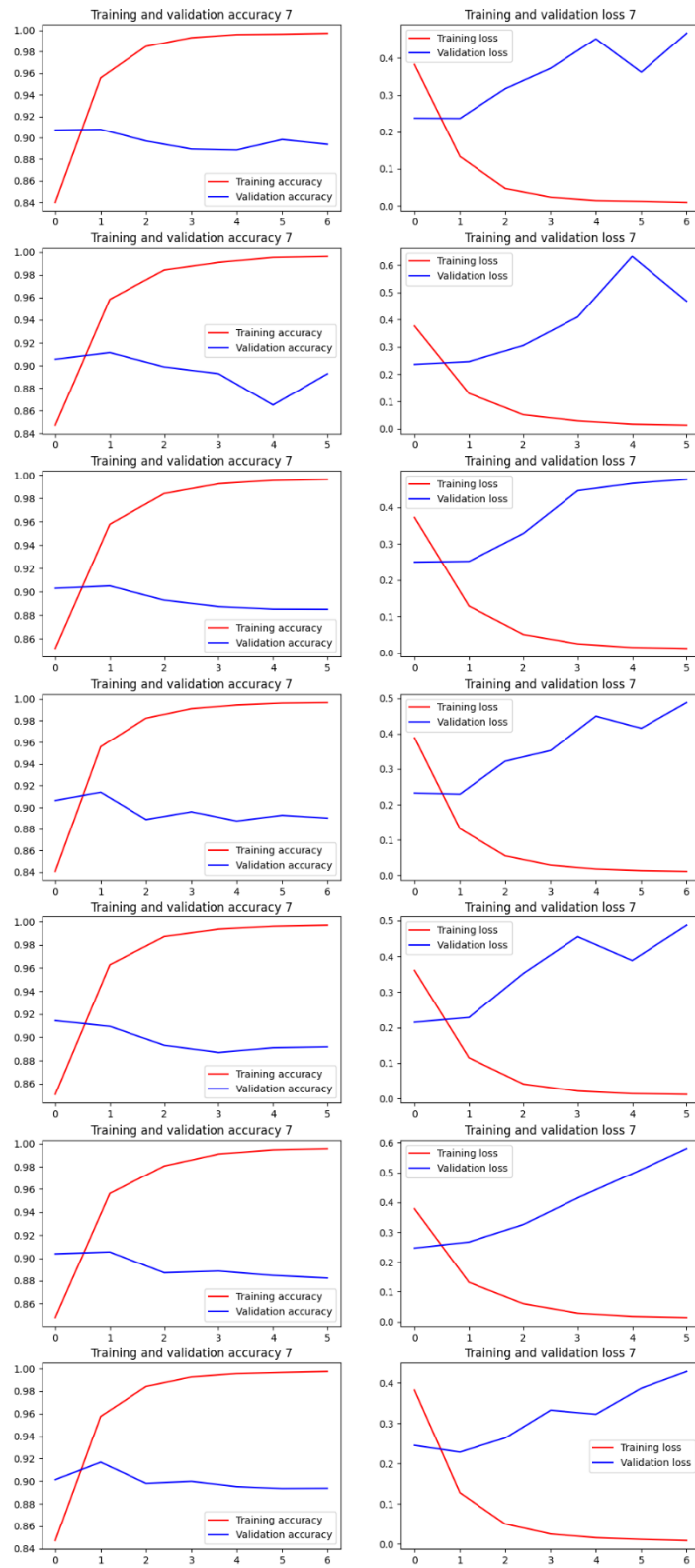
Lampiran 20 Grafik akurasi dan loss tiap iterasi 3-fold



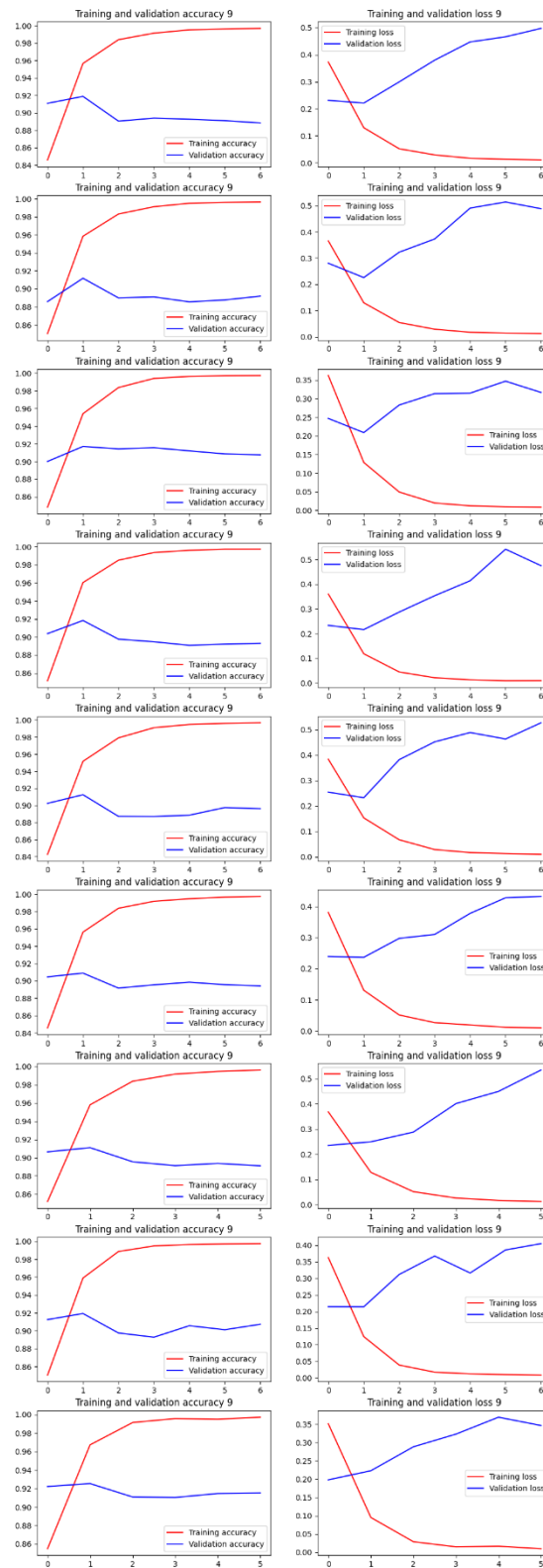
Lampiran 21 Grafik akurasi dan loss tiap iterasi 5-fold



Lampiran 22 Grafik akurasi dan loss tiap iterasi 7-fold



Lampiran 23 Grafik akurasi dan loss tiap iterasi 9-fold



Lampiran 24 Error yang didapat tiap run

```
Node: 'Adam/StatefulPartitionedCall'  
Failed to load in-memory CUBIN (compiled for a different GPU?): CUDA_ERROR_OUT_OF_MEMORY:  
out of memory  
[[{{node Adam/StatefulPartitionedCall}}]] [Op:__inference_train_function_3392948]
```